

MANDARIN: Secure User-friendly Blockchain Modular Wallet OS

Seongjin Kim*
mm0ck3r@korea.ac.kr
Korea University
Seoul, South Korea

Sanguk Yun*
dnwjddld94@korea.ac.kr
Korea University
Seoul, South Korea

Jungho Jang*
jj030116@korea.ac.kr
Korea University
Seoul, South Korea

Abstract

This paper presents MANDARIN, a secure and user-friendly modular blockchain wallet OS that addresses persistent private-key exposure and poor multi-chain usability in existing wallet systems. MANDARIN anchors key generation and transaction signing inside ARM TrustZone using OP-TEE, ensuring that private keys never leave a hardware-isolated secure world even under a compromised Android environment. Unlike prior enclave-based wallets limited to single blockchains, the system decomposes wallet logic into per-chain Trusted Applications (TAs) that can be dynamically installed, updated, or removed without firmware reflashing. A Subkey-based delegated signing hierarchy enables scalable and device-scoped authorization of TAs while avoiding global signing-key compromise. The architecture separates concerns across MINIAPP (UI), CA (native client), and TA (secure execution), enabling extensibility without inflating the trusted computing base. MANDARIN integrates an Android-based runtime that orchestrates module lifecycle management while strictly mediating access to secure signing operations via OP-TEE. A WalletStore backend finalizes TA artifacts using secure signing, delivering device-complete modules that can be verified at secure-world load time. The system is implemented on an STM32MP257F-EV1 platform running AOSP with OP-TEE, demonstrating practical deployability on Android-class hardware. Experimental validation confirms correct ECDSA signing behavior and integrity-preserving communication between the Normal World and Secure World. Overall, MANDARIN demonstrates that a modular TEE-based wallet OS can simultaneously achieve strong key isolation, scalable multi-chain support, and real-world usability.

Keywords

Blockchain, Trustzone, wallet, Trusted computing, Domain-specific security and privacy architectures, Authentication, Mobile platform security, Embedded systems security

1 Introduction

1.1 Historical Analysis of Major Exchange Breaches

1.1.1 2014 Mt.Gox. The 2014 Mt. Gox hacking incident is widely regarded as one of the most pivotal turning points in the history of cryptocurrency security [1, 2]. It illustrated how technical vulnerabilities, operational deficiencies, and an immature market environment converged to produce catastrophic outcomes. At the time of the breach, Bitcoin was transitioning from a speculative novelty into a financial asset, yet exchanges lacked the systematic security standards necessary to cope with a rapidly expanding user base. Although Mt. Gox wielded significant market influence, it

failed to establish robust IT infrastructure and internal control procedures, and lacked monitoring mechanisms capable of detecting and reconciling discrepancies between ledger records and wallet reserves [3]. Subsequent investigations reported that the attackers exfiltrated funds gradually over an extended period, masking unauthorized withdrawals within normal transaction flows—indicating not a single-point failure, but a prolonged infiltration compounded by systemic operational weaknesses, reminiscent of traditional financial sector failures.

Mt. Gox relied on proprietary software and maintained a closed operational model that excluded external developer scrutiny or third-party code auditing [2]. This insularity delayed vulnerability discovery and fostered an environment where problems remained unnoticed until the losses had grown severe. The exchange also lacked personnel with sufficient expertise to manage both technical and accounting issues, while external audits and regulatory oversight were effectively nonexistent [4]. The inquiry revealed inadequate bookkeeping, improper backup and log management, and the absence of an incident-response plan—all of which hindered early detection and contributed to confusion and lack of transparency during public disclosure.

The repercussions extended beyond technical domains, influencing market structure, user behavior, regulatory development, and security research. In response, the industry witnessed growing demand for decentralized wallets, cold storage, and hardware-based key protection [1]. The Mt. Gox incident also catalyzed regulatory intervention; Japan’s Financial Services Agency (FSA) began developing registration systems and audit requirements for crypto-asset service providers [4]. Academic and media discourse frequently characterize the event as a milestone marking the maturation of the cryptocurrency industry, illustrating how centralized architectures and trust dependencies expose systemic risk.

From a technical standpoint, Mt. Gox accelerated the adoption of practices that were previously absent or informal—such as supply-chain verification, proof-of-reserves, multi-signature wallets, segregated storage, and external auditing [3]. Its opaque operational model renewed interest in open-source security analysis, community-driven vulnerability assessments, and bug-bounty programs. Merkle-tree-based proof-of-reserves proposals emerged as a means of improving asset transparency, while principles such as wallet segregation, log auditing, and secure key management began to crystallize into industry norms.

In conclusion, the Mt. Gox hack was not merely a criminal incident; it exposed the structural immaturity of the early cryptocurrency ecosystem and triggered a re-evaluation of technical, operational, and regulatory security frameworks [2]. Ultimately, the incident underscored the importance of private-key protection,

*All authors contributed equally to this research.

internal controls, and external verification, establishing foundational principles that continue to shape modern wallet security and exchange-operation standards.

1.1.2 2018 Coincheck. The 2018 hacking incident targeting Japan's Coincheck exchange is widely regarded as a vivid illustration of the security risks inherent in centralized cryptocurrency platforms, occurring at a time when digital assets were beginning to enter mainstream regulatory discourse [5]. Although Coincheck had grown into one of Japan's largest exchanges in terms of transaction volume, vulnerabilities in its asset custody strategy—particularly the lack of clear separation between hot and cold storage—ultimately enabled attackers to steal a substantial amount of funds [6]. By breaching the exchange's operational infrastructure, the adversaries were able to gain direct access to internal wallets and bypass authentication and monitoring mechanisms, exposing the systemic risks of maintaining customer assets in online storage. The prolonged absence of segregated storage policies and multi-signature protections amplified the scale of the compromise.

The significance of the Coincheck incident lies not merely in the monetary value of the stolen assets, but in the fact that the breach was executed through relatively simple means, yet resulted in massive real-world damage [5]. It highlighted how cryptocurrency security hinges less on cryptographic strength and more on operational safeguards such as private-key management, access control, and system governance. The fact that keys were managed and stored in networked environments demonstrated that large-scale digital theft can occur without physical intrusion, necessitating a risk framework distinct from traditional financial systems. Coincheck's delayed crisis response further exposed weaknesses in transparency and incident management, illustrating deficiencies in risk planning and coordination under Japan's regulatory posture at the time [4].

The incident had a far-reaching impact on both Japanese regulators and the global cryptocurrency market. Japan's Financial Services Agency (FSA) strengthened registration and oversight requirements for exchanges and ordered Coincheck and other major platforms to enhance their security controls and internal risk-management policies [4]. Key protection mechanisms—including segregated hot/cold storage, adoption of multi-signature wallets, and stricter access controls—effectively became mandatory. From a user perspective, the attack fueled skepticism toward centralized custody models and accelerated demand for hardware wallets, cold storage, and self-custody alternatives [5].

From a research standpoint, the Coincheck compromise underscored that cryptocurrency exchange security extends far beyond software bugs. It emphasized supply-chain risks, the necessity of internal auditing, rigorous access-control policies, and proactive network-intrusion detection. The theft of NEM (XEM) tokens also revealed how breaches can propagate across the network layer, affecting market stability and asset valuation, demonstrating the need for asset-specific threat modeling and coordinated mitigation strategies [6]. Ultimately, the case revealed that the vulnerabilities of the industry were not solely technical, but were deeply intertwined with operational mismanagement, regulatory immaturity, and insufficient risk planning.

In summary, the 2018 Coincheck hack demonstrated that centralized exchanges cannot sustain user trust without robust operational security policies governing asset custody, private-key management, access control, and incident response. The event served as a critical inflection point, broadening the security discourse beyond mere technical defenses and into governance, oversight, and transparency [4]. It remains a landmark example of how operational failures can undermine the core trust assumptions of centralized cryptocurrency ecosystems.

1.1.3 2022 FTX Bankruptcy and Internal Hacking Case. The 2022 collapse and subsequent internal asset drain at FTX is widely regarded as a landmark case demonstrating that even in a mature phase of the cryptocurrency market, centralized exchanges remain vulnerable to rapid implosion when operational governance and internal controls fail [7]. Prior to its downfall, FTX had achieved global prominence through high liquidity, aggressive marketing, and a broad suite of financial products; yet the incident revealed that this outward success masked opaque asset management practices and a fundamentally deficient risk-management framework [8]. Bankruptcy proceedings later confirmed that customer deposits were commingled with proprietary funds and used in transactions involving affiliated entities, while publicly reported financial statements diverged significantly from actual reserve conditions [9].

A particularly notable aspect of the incident is that the asset drain is widely believed to have been executed by an insider rather than external attackers [10]. This underscores the necessity of controlling internal actors—not merely defending against external adversaries—in centralized exchange architectures. The unauthorized transfers exploited the chaos immediately following the bankruptcy filing, demonstrating that inadequate privilege management and key control mechanisms allowed a single insider to effectuate large-scale fund movements. The event starkly illustrated the importance of internal controls and segregation of duties—principles long established in traditional finance—within the digital asset ecosystem [11].

The FTX scandal triggered heightened regulatory scrutiny and a sharp erosion of public trust across the broader market. Following the incident, major regulatory bodies—including the U.S. Securities and Exchange Commission (SEC), Commodity Futures Trading Commission (CFTC), and international supervisory agencies—called for strengthened standards regarding customer asset segregation, reserve verification, operational oversight, and comprehensive risk governance [9, 11]. The bankruptcy proceedings further revealed that formal compliance frameworks, when lacking robust enforcement and auditing mechanisms, are insufficient as mere procedural artifacts. These revelations accelerated discussions on the systemic risks inherent in centralized custody, contributing to increased interest in decentralized finance (DeFi) and self-custody alternatives [7].

From a security research perspective, the FTX case demonstrated that critical threats within digital-asset infrastructure need not originate from complex software exploits. Insufficient key management, absent surveillance mechanisms, and concentration of administrative privileges can enable catastrophic breaches independent of traditional hacking [8]. The incident highlighted the dangers of centralized control over root keys and administrative authority

and emphasized the importance of key distribution, role-based access control (RBAC), multi-signature schemes, audit logging, and rigorous monitoring. Additionally, the challenge of tracing unauthorized asset flows during bankruptcy exposed limitations in cross-jurisdictional enforcement, real-time blockchain analytics, and regulatory-technical coordination [10].

In summary, the FTX collapse and internal asset drain did not merely represent a bankruptcy or theft incident; it revealed the systemic consequences that arise when centralized cryptocurrency operators lack governance, transparency, and robust oversight. The event reaffirmed that cryptographic defenses alone cannot secure exchanges in the absence of rigorous internal control, financial integrity, segregated custodial models, and comprehensive reporting mechanisms [11]. In this respect, the FTX scandal echoes and extends the lessons raised by earlier incidents such as Mt. Gox and Coincheck, and stands as a catalyst for re-examining dependency on centralized service architectures within contemporary cryptocurrency markets.

1.1.4 2024 DMM Bitcoin hack. The 2024 hacking incident at Japan's DMM Bitcoin exchange demonstrated that large-scale digital asset theft remains possible even in regulatory environments that have reached a mature stage [12]. Despite Japan having instituted robust supervisory frameworks—particularly following the Coincheck breach—and having formally mandated segregated custody and wallet separation principles through the Financial Services Agency (FSA) [13], the attack revealed that centralized exchanges remain susceptible when weaknesses emerge in specific operational designs or real-time monitoring practices. Post-incident analysis indicated discrepancies in the security posture applied to multi-signature-protected assets versus those managed through single-custody schemes, leaving certain holdings disproportionately exposed [14]. The attack was executed via rapid, high-volume withdrawals, leveraging vulnerabilities in asset-transfer channels and network infrastructure rather than targeting the transaction system itself, thereby evading existing detection mechanisms.

A central takeaway from the DMM Bitcoin case is that neither regulatory maturity nor compliance alone can fully eliminate technical and operational vulnerabilities. Although the exchange adhered to legal requirements governing asset segregation, hot-cold wallet separation, backup procedures, and multi-factor authentication, attackers were nevertheless able to exploit specific custody paths and transfer funds within a short timeframe [12]. This illustrates that formal compliance is insufficient as a standalone defense; effective protection requires adversarial-perspective modeling, real-time traffic anomaly detection, and continuous monitoring of asset flows.

The incident had repercussions beyond the exchange itself. The FSA initiated an urgent audit and mandated a reassessment of internal security controls and access-management policies [13]. While DMM Bitcoin pledged to reimburse customer losses using its own funds, this response highlighted governance concerns regarding the socialization of risk when centralized exchanges absorb the financial impact of external security failures. The case also underscored the need for regulatory frameworks to expand beyond reserve assurance and custody segregation, toward the inclusion of operational controls such as incident-response planning, crisis communication structures, and internal information-leak prevention [14].

From a research perspective, the attack prompted reconsideration of hybrid security models that span both on-chain traceability and off-chain key-management infrastructure. The breach showed that even with multi-signature architectures, inconsistent security enforcement across different asset types or wallet nodes can create exploitable vectors [15]. This emphasizes that security cannot be differentiated based on asset categories and that uniform protection levels must be maintained across the entire custody portfolio. Furthermore, the event reinforced the importance of proactive penetration testing, automated monitoring systems, and continuous intrusion-detection capabilities—highlighting that centralized exchanges must not rely solely on external audits but must actively verify the resilience of their internal infrastructure [15].

In conclusion, the 2024 DMM Bitcoin breach demonstrates that regulatory frameworks alone cannot guarantee resilience if operational readiness and security maturity remain insufficient [12]. The incident underscores the need for centralized exchange security strategies to evolve beyond compliance toward proactive control of asset flows, distributed authorization mechanisms, real-time monitoring, and enterprise-wide risk assessment. As such, it represents a pivotal moment in Japan's post-Coincheck regulatory landscape, emphasizing new research imperatives around adversarial threat modeling, operational security, rapid response, and improvements to custodial segregation models.

1.1.5 2025 Bybit Exchange Hack. In February 2025, the Dubai-based cryptocurrency exchange Bybit suffered what is widely considered the largest theft in the history of digital asset trading platforms, with attackers stealing approximately \$1.4–1.5 billion worth of Ethereum and staked ETH tokens [16]. Reports indicate that around 400,000 units of ETH and stETH were illicitly transferred from the exchange's custody, making this incident the most sizable single loss ever recorded in the cryptocurrency sector.

The breach occurred on February 21, 2025, when sophisticated attackers compromised Bybit's multi-signature cold wallet infrastructure. Rather than exploiting a simple software flaw, the attack appears to have stemmed from a third-party dependency in the multi-signing workflow, specifically involving the Safe{Wallet} platform used by Bybit for transaction authorization [17]. Investigative sources indicated that the attackers employed social engineering and operational compromise techniques to manipulate authorized signers into approving fraudulent transactions.

Law enforcement and blockchain analysis organizations quickly linked the incident to advanced persistent threat actors associated with the Lazarus Group, widely believed to operate on behalf of North Korea. Official statements from the Federal Bureau of Investigation (FBI) characterized the attack under the codename "TraderTraitor," attributing it to coordinated DPRK-backed cyber operations [18]. The FBI also warned that stolen assets were being moved across multiple blockchain addresses, with laundering attempts expected.

The immediate market impact was significant, with major cryptocurrencies reacting sharply to the news. Ether and Bitcoin prices experienced notable declines as confidence in centralized exchange security was challenged. In response, Bybit acknowledged the loss and described efforts to replenish reserves through emergency funding provided by liquidity partners. The exchange also initiated a

public bounty program in an effort to trace and freeze stolen funds [19].

This incident underscores evolving threat landscapes where state-level actors target not only DeFi protocols but also centralized custodial infrastructure. Analysts have highlighted that the compromise of a multi-signature wallet—historically regarded as a strong security primitive—reveals how operational dependencies and third-party integrations can be weaponized against centralized platforms [20]. Furthermore, the scale of the theft contributed to an unprecedented share of 2025’s cumulative crypto crime statistics, with Bybit alone accounting for a substantial proportion of global stolen value.

In sum, the 2025 Bybit hack exemplifies how even large, globally significant exchanges remain vulnerable to complex threat vectors that combine social engineering, supply-chain risk, and state-sponsored intrusion. The attack prompted renewed scrutiny of custodial design practices, particularly around multi-signature wallet governance, internal access controls, and third-party service dependencies. Bybit’s experience has consequently stimulated broader discussions about enhancing centralized exchange security architectures to withstand high-impact breaches in the future.

1.1.6 2025 Upbit Hack. In November 2025, Upbit—South Korea’s largest cryptocurrency exchange—experienced a major security breach that resulted in the unauthorized withdrawal of approximately 44.5 billion won (roughly USD 30 million) worth of digital assets [21]. The incident occurred within a narrow window of time and primarily targeted Solana-based hot wallet infrastructure, where attackers executed rapid, large-volume transfers that went undetected by Upbit’s automated risk-control mechanisms. Although the exchange had previously strengthened its security posture following earlier incidents and operated within one of the world’s most mature regulatory environments, the breach exposed persistent vulnerabilities inherent to hot-wallet management and real-time asset-handling workflows [22].

Investigations by South Korean authorities—including the Financial Supervisory Service (FSS), the Korea Internet & Security Agency (KISA), and national law-enforcement units—focused not only on the technical execution of the theft but also on Upbit’s disclosure timeline and internal control procedures [23]. Notably, several hours passed before regulators were formally notified, prompting criticism regarding incident-reporting obligations under domestic oversight frameworks. Subsequent media reports and intelligence assessments suggested possible links to the North Korea-aligned Lazarus Group, reflecting recurring patterns in state-sponsored attacks targeting cryptocurrency services [24].

Despite the scale of the loss, Upbit committed to fully reimbursing affected customers using its own reserves, limiting direct financial harm to users while simultaneously highlighting governance and moral-hazard considerations associated with centralized custodial models [25]. The incident triggered broader policy discussions in South Korea concerning exchange liability, reserve transparency, and the adequacy of existing supervisory mechanisms. Regulators signaled plans to reinforce compensation mandates and expand real-time monitoring requirements, illustrating the tension between compliance-oriented frameworks and operational security realities.

Ultimately, the 2025 Upbit breach demonstrated that even well-regulated and technologically sophisticated exchanges remain susceptible to targeted attacks when operational monitoring and hot-wallet safeguards are circumvented. The event reaffirmed that compliance alone cannot guarantee resilience and underscored the need for continuous auditing, automated anomaly detection, and stricter controls over hot-wallet infrastructure [22]. As such, the incident serves as a contemporary example of the systemic risks faced by centralized custodial platforms, reinforcing lessons drawn from earlier cases while raising new questions about regulatory preparedness and exchange accountability in an evolving threat landscape.

1.2 Motivation

Over a decade of rapid innovation in cryptocurrencies has failed to resolve a surprisingly persistent reality: the overwhelming majority of digital-asset losses stem not from esoteric cryptographic failures or consensus-level attacks, but from the compromise of private keys themselves [26]. Industry intelligence firms continue to document this imbalance at scale. Chainalysis’ 2024 review illustrates that asset theft attributable to compromised credentials reached multi-billion-dollar magnitudes, reflecting an upward trajectory rather than a downward trend. Rather than being isolated incidents, such losses represent a structural weakness in present key-management practices and reveal that academic progress has yet to translate into consistent, real-world security.

A closer examination of incident reports demonstrates that private-key exposure remains the single most consequential attack vector across diverse wallet and custody models [27]. Whether obtained via malware, social engineering, backdoored firmware, supply-chain infiltration, or coercion, the theft of signing authority enables adversaries to bypass virtually all blockchain-level protections. This is not an artifact of immature projects or fringe networks: mainstream platforms and well-capitalized services have repeatedly fallen victim. Centralized custodians, despite employing dedicated security teams and audited infrastructure, continue to face concentrated risks because the authentication credentials for massive pools of assets invariably converge on single points of compromise [28]. The now-infamous wallet drain incidents—some escalating to extortion or physical threats—underscore that private-key compromise carries severe consequences beyond digital boundaries.

Defenses traditionally perceived as “safe,” including cold wallets and multi-signature arrangements, are no longer immune in practice [29]. The trend of adversaries targeting the environment in which keys are generated or imported has introduced new forms of exploitation. Hardware components used during setup or signing—ranging from USB microcontrollers to trusted displays—have become attack surfaces through which keys or seeds can be leaked even before they reach an enclave or cold device. This evolution highlights that physical separation alone does not absolve custodians of exposure risks when upstream supply chains and deployment environments remain unverified.

Self-custody approaches, widely promoted as the antidote to centralized vulnerabilities, offer only conditional protection. Hardware wallets mitigate persistent storage threats, yet their interaction with

online hosts during transaction signing returns the user to a vulnerable execution context [30]. Connectivity re-introduces exposure to browser extensions, rogue drivers, DMA injection, keyloggers, phishing portals, and more. Attempts to eliminate this interface—by resorting to air-gapped models or offline backups—ultimately defer risk rather than eliminating it. The signing operation inevitably transits through a computing engine capable of arbitrary code execution, enabling side-channel leakage, fault injection, or memory scraping during the narrow temporal window in which private keys must be in plaintext to authorize transactions.

Given these realities, researchers have naturally explored hardware-backed isolation domains such as ARM TrustZone or Intel SGX as a more resilient alternative [31, 32]. And while enclave-based wallets provide meaningful separation between secret keys and untrusted operating systems, their current form exhibits critical shortcomings. Many are tightly bound to a single protocol or blockchain, with designs tailored specifically for Bitcoin, Ethereum, or analogous systems [33]. This specialization simplifies implementation but sacrifices broader applicability and forces users with diverse portfolios to juggle fragmented toolchains.

This fragmentation produces ripple effects at an ecosystem level. Newer or niche blockchains lack timely enclave-supported wallet implementations, relegating early adopters to web-based hot wallets or unverified binaries [34]. Moreover, the auditing community faces duplicated effort: each isolated implementation must be validated independently, often reviewing similar cryptographic routines multiple times with no cross-module benefit [35]. This inefficiency increases cost and risks inconsistent assurance levels.

Taken together, the present state of wallet security reveals two overarching deficiencies. First, contemporary mechanisms fail to reliably safeguard private keys throughout their operational lifetime, especially during the signing phase—a moment where all abstractions converge and the user’s most sensitive asset is momentarily exposed. Second, the fragmentation of tooling, protocols, and enclave deployments demonstrates a persistent inability to accommodate multi-chain ownership in a scalable way [36]. These parallel shortcomings hinder both individual users and the broader ecosystem.

The cumulative evidence therefore indicates a critical inflection point: protecting private keys cannot be framed solely as a choice between centralized custody and isolated hardware wallets, nor can enclave-based security be considered sufficient if it remains tied to monolithic, chain-specific deployments. What is currently lacking is a unified perspective that acknowledges real-world usage patterns—where portfolios span many chains, keys must be used across heterogeneous applications, and threat actors exploit both technical and social vectors. Any meaningful advance in key-management must address not only secrecy but also extensibility and operational practicality to prevent the systemic failures observed in contemporary infrastructure.

1.3 Our Approach

1.3.1 Design Objectives. The overarching ambition of this research is to construct a wallet substrate capable of safeguarding cryptographic keys throughout their entire lifecycle, while concurrently

supporting heterogeneous blockchain ecosystems without sacrificing practicality or openness. To steer the system toward this vision, we identify four guiding principles:

(1) **End-to-end key isolation:**

All private-key generation, storage, and signing are confined to the ARM TrustZone Secure World [31, 37]. By ensuring that keys never traverse the Normal World memory hierarchy, we suppress avenues such as kernel-level instrumentation, page-table manipulation, and DMA snooping [32], thereby limiting the exposure window to zero within REE-space.

(2) **Modular execution architecture:**

Instead of bundling logic for multiple protocols into a single Trusted Application, each blockchain network is encapsulated within its own discrete TA. Modular TEE applications have been demonstrated to reduce attack surface area and promote minimality [38]. This decomposition yields smaller attack surfaces per module, promotes independent development cycles, and removes the complexity of monolithic builds.

(3) **User-controlled feature extensibility:**

The wallet framework should empower end-users—not vendors—to select which chains they interact with. This reflects usability and decentralization priorities seen in multi-chain adoption studies [39]. Support for emerging or low-market-cap networks must be deployable immediately and uniformly, without compelling users to migrate devices or adopt inconsistent interfaces.

(4) **Transparent and reproducible trust boundaries:**

The platform must enforce rigorous validation of TA binaries before they are accepted into the Secure World. Verified-provenance workflows and binary attestation—practices increasingly employed in secure-supply-chain systems—provide this assurance [40, 41].

1.3.2 Architectural Blueprint. Figure 2 outlines the layered organization of the platform. At its core, three logical elements collaborate to deliver secure operation:

(i) *Wallet Platform within the REE.*

- Built atop a lightly adapted Android environment, the system exposes a constrained interface for Secure Monitor Calls [37].
- The Client Application (CA) transforms user-initiated actions (seed imports, transaction creation, or address enumeration) into standardized Secure World requests.
- Interface surfaces are minimized to reduce ways in which malformed inputs could attempt to bypass isolation boundaries, consistent with TCB minimization principles [42].

(ii) *Trusted Execution Environment.*

- Each supported blockchain protocol corresponds to a standalone TA binary. Prior research demonstrates the benefits of per-service isolation in enclave deployments [33].
- Protocol-specific concerns—derivation paths, serialization formats, and signing primitives—reside strictly within TA boundaries [43, 44].

- The absence of shared memory or cross-module messaging restricts lateral movement in the presence of compromised modules [45].

(iii) *Module Management Infrastructure.*

- A dedicated backend repository organizes TA submissions, versioned interfaces, and metadata declarations.
- Developers provide manifests and attestations describing supported chain semantics and interface revisions, consistent with emerging reproducible-build policies [46].
- Automated analysis ensures that modules conform to the TEE’s expected calling conventions and resource-allocation policies [37].

In this schema, the REE orchestrates interaction, the Secure World executes sensitive operations, and the repository establishes provenance and repeatability—each layer aware of the others but never entrusted with key material.

1.3.3 Module Lifecycle. The pipeline for incorporating a new blockchain module progresses through distinct checkpoints:

- (1) **Creation:** A third-party developer authors a TA implementing a novel chain. The binary adheres to expected interface specifications [37].
- (2) **Interface Validation:** Submitted binaries undergo static inspection for boundary conformance (buffer limits, capability restrictions), ensuring they behave as true plug-in components rather than privileged extensions.
- (3) **Registration:** Accepted modules are indexed in the repository, annotated with identifiers and compatibility notes, modeled after secure artifact registries [40].
- (4) **Deployment and Activation:** When a user enables a chain, the CA retrieves the TA and transports it into Secure World memory. The TEE performs integrity verification before mapping the binary into a secure address space [31].
- (5) **Revision Handling:** Updates replace prior versions only after re-validation, preventing rollback attacks [47].

1.3.4 User-Level Interaction and Operational Attributes.

- **Unified Environment:** Users perceive the system as a single wallet, though each protocol remains isolated—aligning with multi-chain usability studies [39].
- **Controlled Resource Footprint:** Unused chains impose no penalty; users enable only what they require.
- **Migration Support:** Integration of BIP-32/39 seeds mirrors established hierarchical deterministic design [43, 44].
- **Minimized Custodial Liability:** Server components act solely as registrars and validation engines—never handling private keys.

1.3.5 Key Contributions (Rephrased & Expanded).

- (1) **Decoupled TEE execution model:** We transform monolithic OP-TEE deployments into flexible multi-tenant module systems, enabling compartmentalization and parallel chain logic [38].
- (2) **Secure module procurement and verification pipeline:** Strong binary attestation and validation practices reduce risks inherent in third-party chain support [40, 41].

- (3) **Democratized extensibility for emerging networks:** New blockchains can be integrated without modifying base firmware, promoting community contribution.

1.3.6 Synthesis. Rather than proposing a single-purpose enclave wallet, this work reframes wallet security as an extensible platform service. By confining key material exclusively to the Secure World and enabling chain-specific logic to be composed as modular TAs, our approach reconciles two competing demands: strong isolation of keys and scalable support for heterogeneous blockchain ecosystems. The architecture embodies a shift from rigid, one-chain-per-device paradigms toward a composable, user-centric model, where interoperability and confidentiality are achieved simultaneously without reliance on firmware-wide updates.

2 Background

2.1 ARM TrustZone

ARM TrustZone is a system-wide hardware security technology that partitions the system-on-chip (SoC) into two isolated execution environments: the Secure world and the Normal world (Figure 1)[48]. In a TrustZone-enabled processor, sensitive code and data run in the Secure world, which has access to secure hardware resources, while the Normal world hosts the rich operating system and applications and is barred by hardware from accessing secure resources[48]. This division is enforced at the hardware level – the CPU, memory, bus, and peripherals are all tagged as secure or non-secure, and the TrustZone logic ensures that Normal-world software cannot read or write Secure-world memory or devices[49]. For example, the system bus includes a special “Non-secure” (NS) signal bit to label transactions, and memory controllers will block any access from the Normal world to regions marked as secure[49]. The Secure world, however, is not so restricted and can (if needed) access Normal-world memory, enabling monitored interaction between the two domains[49]. By implementing isolation in hardware, TrustZone makes it significantly more difficult for malicious or compromised code in the Normal world to affect critical assets in the Secure world, yielding a stronger security foundation than purely software-based sandboxing[50].

A single physical processor core can time-slice between the Secure and Normal worlds, giving each an illusion of exclusive use[49]. Context switches between worlds occur via a special privileged mode (the Secure Monitor at ARMv7 or EL3 at ARMv8) using a secure monitor call (SMC) instruction or interrupts[49]. This allows a high-performance “rich” OS to run in the Normal world (e.g. Android or Linux) while a lightweight trusted kernel runs in the Secure world[49]. TrustZone systems employ a secure boot process to initialize this split world: early bootloaders and the Secure-world OS are cryptographically verified, establishing a chain of trust that ensures only authenticated code executes in the Secure world[49]. Once the device is booted, any request from a Normal-world application to perform a sensitive operation (like using a cryptographic key or accessing a biometric sensor) can be routed via the secure monitor into the Secure world for processing by a trusted service, with the hardware ensuring isolation throughout.

TrustZone’s hardware-enforced isolation enables the concept of a Trusted Execution Environment (TEE) as standardized by

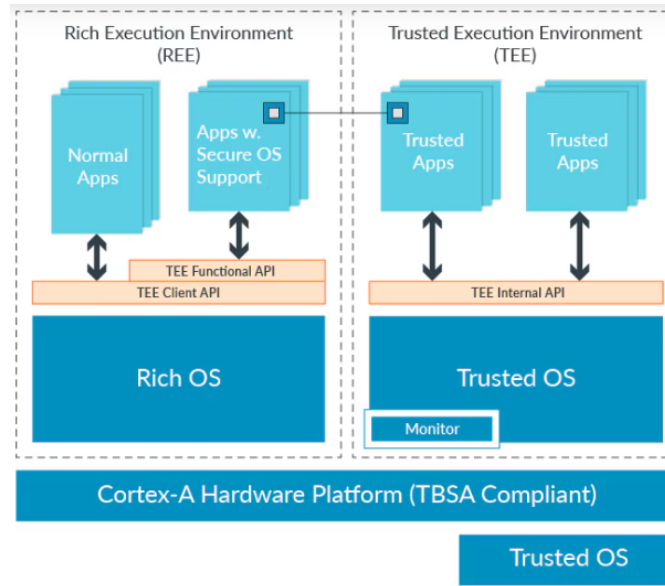


Figure 1: Overall architecture of ARM TrustZone

GlobalPlatform[50]. In practice, this means critical security functions can be offloaded to the Secure world. For instance, modern smartphones use TrustZone to protect sensitive operations like password/PIN verification, mobile payments, and biometric authentication by running those tasks in the Secure world away from the main Android OS[50]. By design, even if the Normal-world OS is compromised by malware, the Secure world remains walled off and continues to protect cryptographic keys and security-critical code. These advantages – hardware-backed isolation and one-way access control – make TrustZone a foundation for mobile and IoT security, enabling a secure OS to run in parallel with the normal OS on the same hardware without additional processors[49].

2.2 OPTEE

OP-TEE (Open Portable Trusted Execution Environment) is an open-source Trusted OS that runs in the Secure world of ARM TrustZone, providing a reference implementation of a TEE in compliance with GlobalPlatform specifications[51]. OP-TEE consists of a small secure kernel (running at the EL1 secure privilege level on ARMv8 or S-mode on ARMv7) along with a set of internal APIs for developing Trusted Applications (TAs) that execute in the Secure world[52]. It implements the GlobalPlatform TEE Internal Core API for secure services and the TEE Client API for communication with Normal-world software[52]. Correspondingly, OP-TEE’s architecture includes a Normal-world component (the OP-TEE client) comprised of a kernel driver and user-space daemon (“tee-supplciant”), which together handle the interface between the rich OS (e.g. Linux/Android) and the secure OP-TEE OS[52]. When a Normal-world application – also called a Client Application (CA) – needs to perform a secure operation, it invokes the TEE Client API (e.g. using the libteec library). The OP-TEE driver then uses a Secure Monitor Call (SMC) to trap into the Secure world, where the OP-TEE kernel dispatches the request to the appropriate TA

for handling. This design effectively follows a client-server model with OP-TEE acting as a secure service provider and the Normal-world OS as the client. OP-TEE also provides secure services like cryptographic primitives and sealed storage, so TA developers can focus on application logic while relying on the TEE for security functions.

All Trusted Applications in OP-TEE are digitally signed, and OP-TEE enforces an integrity check on each TA binary before loading it. In the default OP-TEE trust model, there is a single root signing key (a RSA/ECDSA key pair) whose public key is embedded in the TEE core at build time (or in hardware fuses); every TA’s signature is verified against this root public key. Only if the signature is valid (meaning the TA has not been tampered with and was signed by the trusted key) will the OP-TEE OS allow the TA to be loaded and executed. This mechanism ensures that rogue or unauthorized code cannot run in the Secure world. However, the single root key approach has limitations in collaborative environments – if multiple vendors or parties need to develop TAs, sharing one private signing key is undesirable and doesn’t scale[51]. To address this, OP-TEE introduced a Subkey architecture in version 3.20, which extends the TA verification model into a hierarchy of trust[51].

Under the subkey scheme, the TEE can accept a chain of certificates (subkeys) leading from a root of trust to the TA (Figure 2). The root key (still embedded in OP-TEE core) can issue a first-level subkey certificate (essentially a signed public key). That subkey in turn can sign another subkey, and so on, delegating trust down a chain[51]. Finally, a subkey in the chain signs the TA. At load time, OP-TEE will verify the TA’s signature using the last subkey’s public key, then verify that subkey’s signature using the next higher key, and so forth up the chain until ultimately verifying the first subkey with the root public key[51]. In this way, OP-TEE establishes a chain of trust from the root key to the TA, without requiring the root private key to sign everything directly. Different organizations

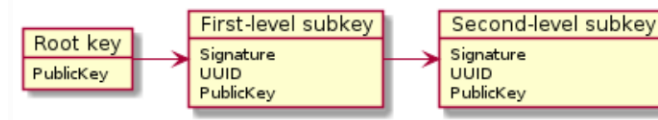


Figure 2: Overall architecture of ARM TrustZone

can be given different subkeys to sign their respective TAs, so that one company’s TA signatures won’t be accepted under another’s subkey, and the root key itself can remain safely offline[51]. Each subkey includes a metadata field defining a UUID namespace – essentially, a restriction that any TA or subkey it signs must have a UUID derived from that namespace[53]. This prevents a delegated subkey from signing TAs outside its intended scope: for example, a subkey granted to vendor A cannot be used (even if compromised) to sign a TA whose UUID belongs to vendor B’s namespace[53]. The TA’s own UUID is checked against the chain of subkey certificates during verification to enforce this constraint. The result is a flexible public-key hierarchy for TA authentication[51]: multiple parties can securely contribute TAs to the system, each using their own signing key protected by OP-TEE’s chain-of-trust.

When a TA is signed using subkeys, the distributed TA package includes the entire chain of subkey certificates prepended to it (except the root, which the TEE already has)[53]. This means the OP-TEE runtime can perform verification without any external dependencies: the TA binary is self-contained with its own cert chain. It will parse the attached subkey certificates in sequence, check each signature and associated UUID namespace, and only if the chain is intact and the final signature on the TA is valid will the TA load succeed[53]. In summary, the subkeys feature extends OP-TEE’s trust model to support delegation and multitenancy in TEE deployments. It preserves TA integrity via a chain-of-signatures, analogous to a certificate chain in PKI, rooted in a hardware/firmware root of trust[51]. This enhances security (by limiting the impact of a single key compromise) and flexibility (by allowing separate management of different TA signing keys) in environments where many stakeholders produce trusted applications for the TEE.

2.3 Blockchain

Blockchain technology provides a decentralized and cryptographically secure way to maintain a ledger of transactions across a network of untrusted participants. In essence, a blockchain is a distributed database (ledger) that is shared among a network of computers, where each addition to the database is bundled into a block, and blocks are chained together sequentially using cryptographic hashes[54]. This structure ensures immutability: once a block is added to the chain, it becomes extremely difficult to alter its contents, because any change would invalidate that block’s hash link to its successor, and the discrepancy would be detected by other nodes[54]. Every block contains a cryptographic hash of the previous block, creating an interdependent chain from the first block (genesis) to the most recent. Consequently, if an adversary tries to tamper with a historical record, the hash pointers in all subsequent blocks would no longer match, and the network nodes would reject the forged chain as invalid.

Blockchains achieve decentralized trust through replication and consensus. Instead of a central authority maintaining the ledger, every network participant (node) keeps a copy of the blockchain and updates it in accordance with agreed-upon rules[54]. When new transactions occur, they are grouped into a new block which must be validated by the network (for example, by consensus algorithms like Proof of Work or Proof of Stake). Only after consensus is reached on the validity of the block is it appended to the chain, and all nodes update their copies[54]. This process ensures that no single party can unilaterally falsify the ledger; additions are accepted only if a majority (or a required quorum) of the network agrees. Moreover, because all nodes independently verify each block and the cryptographic links secure the history, there is no need to trust any intermediary – trust is placed in the mathematics of hashing and the protocol rules[55]. As a result, blockchain networks can function trustlessly, meaning participants can transact and cooperate without relying on a central broker or mutual trust, as the system itself guarantees integrity and order of records[55].

Once data is recorded on a blockchain and sufficient consensus confirmations have occurred, it becomes effectively permanent. Every block added on top of an entry further cements its place in history, since altering any block would require redoing the proof-of-consensus for that block and all blocks after it on a majority of nodes – an almost insurmountable task for well-secured networks (especially public blockchains like Bitcoin). In this way, blockchain provides immutability and transparency: all validated transactions are timestamped and visible to the network (or to permitted members in a private blockchain), creating an append-only audit trail. These properties make blockchain useful beyond cryptocurrencies. For example, in supply chain management, a blockchain can track goods with guarantees that records haven’t been altered; in decentralized identity, it can prove attributes without a central issuer; and in any scenario requiring a distributed ledger (such as voting, asset registries, or IoT data integrity), blockchain offers a way to harden the system against tampering and single-point failure[55]. Importantly, the security of a blockchain comes from a combination of cryptographic techniques (hashing, digital signatures) and game-theoretic incentives in the consensus mechanism, which together ensure that maintaining an honest ledger is far more economically viable than attempting to cheat it.

2.4 Android Open Source Project (AOSP)

The Android Open Source Project (AOSP) is the foundation of the Android operating system in its pure, open-source form. It includes the entire core software stack for Android devices: at the lowest level a modified Linux kernel (monolithic, with Android-specific drivers and enhancements), atop which sit the hardware abstraction layers, system libraries (Bionic libc, media, graphics,

etc.), the Android runtime (ART) that executes app bytecode, the application framework (Java/Kotlin APIs for developers), and a set of default apps (like Dialer, Settings, etc.)[56]. AOSP is released as free and open-source software (Apache 2.0 license for most of the user-space code, and GPLv2 for the Linux kernel changes)[56]. This means anyone – device manufacturers, developers, researchers – can study the code, build it for their hardware, and modify it to create custom Android distributions. Indeed, AOSP is the common codebase that phone OEMs (Samsung, Google, Xiaomi, etc.) start from to make their branded Android firmware. They typically add proprietary applications and services (for example, Google Mobile Services on certified devices) or custom user interface layers on top of AOSP, but the underlying OS functionality and APIs remain those provided by AOSP[56]. This open model has been key to Android’s widespread adoption: AOSP provides a unified, standard platform, while allowing differentiation – companies can customize or extend the OS to meet their hardware capabilities or design goals without having to reinvent the core OS from scratch[56].

Architecturally, AOSP follows a classic layered design that separates hardware-dependent code from the higher-level OS frameworks[56]. At the bottom is the Linux kernel, which handles low-level tasks like process scheduling, memory management, and device drivers (for Bluetooth, USB, display, etc.)[56]. Above the kernel, Android defines a Hardware Abstraction Layer (HAL): for each hardware component, there is a HAL interface that device vendors implement in C/C++ to allow the Android system services to call into device drivers in a uniform way[56]. This fosters portability – the Android OS can work across different chipsets as long as the HAL contracts are fulfilled. The Binder IPC mechanism provides secure, high-performance inter-process communication between Android services and apps, enforcing security policies (permissions) between isolated processes[56]. On top of these layers sits the Android Runtime (ART), which runs apps compiled to bytecode (DEX format) by translating them into native code (using Just-In-Time or Ahead-Of-Time compilation). ART and the core libraries (Java/Kotlin standard libs, UI frameworks, etc.) constitute the Application Framework – this is what app developers target when building Android applications. Overall, the AOSP architecture is often depicted as a stack: Linux kernel at the base, HALs and native libraries just above, the runtime and framework in the middle, and applications at the top[56]. This modular approach, combined with the open-source nature, makes AOSP highly versatile. It powers not only smartphones and tablets, but also wearables (Wear OS), TVs (Android TV), automotive systems (Android Automotive), and IoT devices, with the same core OS adapted to each domain.

From a security standpoint, Android utilizes the Linux kernel’s security features (process isolation, user-based permissions, SELinux policies) and adds its own multilayered security model on top. AOSP is designed to integrate with TrustZone-based TEEs for handling the most sensitive operations. Notably, Android’s Hardware-Backed Keystore relies on a TEE to securely generate, store, and use cryptographic keys so that even the Android OS cannot directly access raw key material[57]. Google’s reference implementation of a TEE is called Trusty, a small secure OS that runs in TrustZone on Pixel devices and implements the keystore, DRM, and other secure services[57]. In an AOSP build for a given hardware platform, one can replace or augment Trusty with an alternative TEE

such as OP-TEE. Doing so involves integrating the OP-TEE client kernel driver and user libraries into the Android OS, and modifying Android’s TEE services (Keystore, Gatekeeper, etc.) to interface with OP-TEE rather than Trusty. Prior work and implementations have demonstrated Android + OP-TEE coexistence: the Normal-world Android framework communicates with OP-TEE TAs using the same GlobalPlatform client API calls, allowing Android system components or apps to invoke secure-world Trusted Apps for cryptographic operations, authentication, storage, and so on. For example, a banking app might call Android’s Keystore, which under the hood calls into a TA in OP-TEE to perform a signing operation with a private key that never leaves the Secure world. From the app’s perspective, this is seamless – Android provides the API – and from the system’s perspective, the TrustZone hardware and OP-TEE ensure that the key is protected even if the Android OS is compromised. AOSP’s openness makes it attractive for research in such secure integrations: developers can instrument the platform to route certain tasks to a TEE, implement custom TAs, or even apply blockchain-based verification of software components, all on top of a transparent and customizable mobile OS foundation.

2.5 STM32 Evaluation Board

The STM32MP257F-EV1 is an evaluation and development board for STMicroelectronics’ STM32MP257F microprocessor – a hybrid SoC that combines dual Arm Cortex-A35 cores (application processors) with a Cortex-M33 core (microcontroller) on one chip[58]. The two Cortex-A35 CPUs (running up to 1.5GHz) are 64-bit cores capable of hosting high-level OSes like Linux or Android, while the Cortex-M33 (running at 400MHz) is a 32-bit microcontroller core aimed at real-time tasks and deep embedded control. Importantly, the STM32MP257F features Arm TrustZone technology on both its A-class and M-class cores, meaning it supports Secure/Normal world isolation on the Cortex-A35 (for TEEs, etc.) and TrustZone-M security on the Cortex-M33[58]. This enhanced security hardware allows developers to implement a TEE on the A35 side (such as OP-TEE in Secure world and Android in Normal world), and simultaneously use the M33 for time-critical tasks, with TrustZone enforcing separation where needed.

The board comes with a rich set of hardware resources to facilitate development. It includes 4GB of DDR4 RAM (organized as two 16-Gbit DRAM chips) for the processors, a non-volatile memory suite consisting of 512Mbit NOR flash (typically used for bootloader or firmware storage) and a 32Gbit eMMC for OS and data storage[58]. For connectivity and I/O, the STM32MP257F-EV1 provides three 1Gbit/s Ethernet ports, a USB Type-C port (dual-role, supporting power and data), two USB 2.0 Host ports, dual CAN-FD ports for industrial networks, a MicroSD card slot, on-board Wi-Fi/Bluetooth, camera and display connectors (MIPI CSI-2 and DSI/LVDS), and general-purpose expansion headers (Arduino-compatible and Raspberry Pi GPIO header)[58]. An ST-LINK/V3 debugger/programmer is embedded on the board to allow easy debugging and flashing of both the Cortex-A and Cortex-M cores via USB[58]. In summary, the board is equipped to evaluate complex applications that span a Linux/Android OS (on the A35s) and real-time firmware (on the M33), with ample memory and connectivity for IoT, multimedia, and industrial use cases.

The STM32MP257F-EV1 is delivered with software support from ST: notably, a Yocto-based Linux distribution called OpenSTLinux for the Cortex-A35, and the STM32CubeMP2 software package for the Cortex-M33 microcontroller[58]. This means out of the box the A35 cores can run a custom Linux (with drivers for all board features), and developers can write FreeRTOS or bare-metal code for the M33, achieving a heterogenous system. STMicroelectronics also provides an Android Board Support Package, and indeed the STM32MP257 family supports running Android (AOSP) on the Cortex-A35 cores[58]. The ability to run AOSP makes this board suitable for mobile/embedded security research. In our project, we leverage the STM32MP257F-EV1 as a test platform for the combined Android + OP-TEE stack. The dual-core Cortex-A35 is used to run AOSP (Android 13) in the Normal world, while OP-TEE operates in the Secure world on the same cores, taking advantage of the TrustZone support on this SoC. We develop Client Applications (CAs) on Android that communicate with Trusted Applications (TAs) in OP-TEE, effectively reproducing a smartphone-like TEE environment on an accessible development board. This setup allows us to deploy and debug our secure services (which may involve blockchain-based verification and other cryptographic operations) in a realistic hardware scenario. The board's TrustZone enforcement, along with the OP-TEE integration, ensures that the TAs are isolated and protected during execution, while the normal Android environment can still provide a rich interface and connectivity. Using the STM32MP257F-EV1, we can iterate on our secure application, measure performance, and evaluate security in a contained setting before moving to production-grade hardware. In short, the STM32MP257F-EV1 serves as the bridge between theory and practice for our project, enabling full-stack development from hardware all the way to the application layer.

3 Related Works

3.1 TrustZone-backed Bitcoin Wallet (Gentil et al., 2017)

3.1.1 Architecture. Gentil et al. (2017) introduced a TrustZone-backed Bitcoin Wallet (TBW) architecture that leverages ARM TrustZone to partition a Bitcoin wallet into secure and non-secure realms[59]. The wallet's sensitive operations are executed inside the Secure World (using an open-source OP-TEE TEE), while the regular Normal World (e.g. Android OS) handles the user interface and network communication. In this design, the Secure World hosts one or more Trusted Applications (TAs) that implement core wallet functions, and the Normal World client can only invoke high-level commands (like requesting a transaction signature) without direct access to keys. Key architectural features include:

- **Secure Key Management:** Private key generation and storage occur entirely within the Secure World. The seed and keys never leave the TEE, preventing OS-level malware from reading secret material. Secure World also provides a high-quality random number generator for key generation and ECDSA operations.
- **Isolated Transaction Signing:** All transaction signing is performed inside the Secure World TA. Only the resulting signature is returned to the Normal World, ensuring that the OS never sees the raw private key or signing process. This isolation means even

a compromised OS cannot directly forge transactions because it cannot obtain valid signatures without calling the TEE.

- **TEE Secure Storage:** The wallet leverages TrustZone's secure storage (with a persistent file stored in encrypted form) to hold private keys. A write-back cache strategy is used to mitigate performance overhead when writing to secure flash storage, allowing efficient key saving/loading without noticeable lag.
- **Standard Crypto API:** The implementation uses GlobalPlatform TEE APIs for cryptography (e.g. AES, SHA-256, ECDSA) inside the Secure World. This ensures well-vetted cryptographic routines and portability across TrustZone-compliant devices. The entire secure wallet software was built on the open-source BitSafe wallet platform and released openly, enhancing transparency and extensibility.

3.1.2 Limitations. While TBW significantly improved security over traditional software wallets, it has several limitations and design trade-offs:

- **Single-Cryptocurrency Focus:** The TBW architecture was designed specifically for Bitcoin and assumes a Bitcoin-like blockchain model. It does not natively support other cryptocurrencies or blockchains. If users need to manage Ethereum, altcoins, or tokens, separate TAs and changes would be required. This single-network design reflects a common limitation of early TEE wallets – most are tied to one blockchain (e.g., Bitcoin) and lack flexible multi-currency support.
- **Full Node Requirement:** The original TBW implementation targeted a full-node Bitcoin wallet, meaning it stored and processed the entire blockchain locally for security (all blocks and transactions). This leads to large storage and sync time requirements that are impractical on typical mobile devices. As noted by later work, TBW's need to maintain the complete Bitcoin ledger in local storage makes it “not applicable to devices with limited hardware (such as smartphones, tablets, embedded systems)”. In other words, the design favors security at the cost of resource usage, and it was demonstrated on development boards rather than on resource-constrained phones.
- **No Secure User Interface:** TBW does not incorporate a trusted display or input mechanism for user confirmation of transactions. The TrustZone TEE protects keys, but the user still relies on the normal OS's UI to, for example, display the transaction details before approval. A malicious OS could potentially deceive the user by altering what is shown (e.g. showing an innocent recipient address while asking the TEE to sign a different transaction). Unlike dedicated hardware wallets (which have built-in secure screens/buttons), a TrustZone-based wallet on a phone shares the display with potentially compromised software. Thus, TBW cannot fully prevent UI spoofing attacks – it assumes the user can trust the phone's output, which might not hold if the OS is thoroughly compromised.
- **Limited Adoption and Evolution:** As an academic prototype from 2017, TBW served as a proof-of-concept but did not see widespread adoption in popular wallet products. Its design, while open-source, would need continuous maintenance and updates for new Bitcoin protocol features or security patches. The study did not address how to update the TEE application post-deployment.

Without an update mechanism, any vulnerability discovered in the TBW's TA would require users to manually patch or re-install, which is a logistical challenge (the design predates the OTA update schemes discussed in newer works). This static nature means the wallet could become outdated or insecure over time if not actively managed.

3.2 SBLWT: Secure Blockchain Lightweight Wallet based on TrustZone (Dai et al., 2018)

3.2.1 Architecture. SBLWT's design goals center on ensuring the confidentiality, integrity, and authenticity of a mobile Bitcoin wallet's operations even on an untrusted platform[60]. The threat model explicitly assumes the Rich OS and apps may be fully compromised by malware, so the Secure World must protect all critical assets against a hostile host environment. Key security objectives include:

- **Protecting Private Keys and Seeds:** Like other TEE wallets, SBLWT isolates the private key generation, storage, and usage inside TrustZone. The seed phrase (or master private key) is stored in secure persistent memory that the Normal World cannot read. All derivation of child keys and signing of transactions is done in Secure World. This guarantees that an attacker who controls the Android OS cannot directly steal keys or observe them in memory, achieving strong confidentiality for the wallet's most sensitive data. Even if the attacker has kernel-level privileges, they cannot access secure memory or storage due to TrustZone enforced isolation. By preventing key exfiltration, SBLWT aims to secure the user's funds from malware that has been responsible for many thefts in traditional wallets.
- **Ensuring Transaction Integrity (Secure Verification):** A novel goal of SBLWT is to not just protect keys, but also protect the transaction verification process. Traditional SPV wallets still trust the local OS to perform verification or at least to honestly report results, which could be subverted by malware. SBLWT instead brings the verification logic into the TEE: it verifies block headers and Merkle proofs for transactions within Secure World, so that a compromised OS cannot trick the wallet about whether a transaction is confirmed or valid. By encrypting block headers and only decrypting them inside TEE, it also maintains the integrity of blockchain data – if malware tampers with stored headers, the modification will be detected (decrypting gibberish or failing a hash check) and rejected. This ensures the wallet will only sign a transaction (to spend money) if it has independently verified the transaction history needed (e.g., that the inputs are confirmed and belong to the user) in a trusted manner. The integrity of the sync process is protected by securing the transition into the Secure World and by cryptographic verification of data. Thus, even a man-in-the-middle or a malicious OS cannot make the wallet accept a fake block or transaction as real.
- **Secure Boot and Code Integrity:** SBLWT employs a secure bootstrapping mechanism so that from the very beginning of the device startup, the wallet's code is measured and trusted. The goal is to prevent an advanced attacker (with OS control) from backdooring or altering the wallet's TEE code. Using cryptographic signatures to verify the SBLWT Secure OS and app

image ensures that only an authentic, untampered version (as developed by the authors) will run in the Secure World. If malware somehow infects the boot process or normal OS, it still cannot modify the SBLWT program in memory or on disk, because any tampering would break the signature check and the secure boot would fail. This gives users confidence that the TEE wallet code itself has not been compromised by, say, a rootkit. It establishes a Chain of Trust from hardware reset up to the wallet application.

- **Data Confidentiality and Cleanup:** Another goal is to ensure the Normal World learns nothing sensitive even indirectly. SBLWT keeps the wallet address and other data (like which transactions the user is interested in) confidential. It uses Bloom Filters when querying full nodes to avoid exposing the user's addresses to the network in a straightforward way. This preserves privacy against remote nodes. Locally, by encrypting all block headers and by cleaning Secure World memory after use, it prevents the Rich OS from gleaning info such as which blocks or transactions the wallet is dealing with. The context-switch mechanism also ensures no sensitive registers or pointers remain accessible to Normal World on switch-back. Even in terms of side-channel considerations, running the verification in TEE means the OS cannot simply call an API to get addresses or transaction details – it would have to perform a complex side-channel attack to possibly infer that information, which is significantly beyond the capability of typical malware.
- **Availability and Non-Bypassability:** SBLWT's use of NMI for switching is a security measure to guarantee that secure operations always occur when needed. A malicious OS could otherwise try to block calls to the TEE or falsify the request. By relying on a hardware interrupt that the OS cannot mask, SBLWT ensures that whenever the user initiates a transaction or sync, the Secure World will indeed execute the necessary code. This prevents a subtle attack where malware might try to hijack the control flow (e.g., intercept a normal world API call and not actually invoke the TEE, thereby perhaps using a fake signing routine). Instead, SBLWT's architecture forces a genuine world switch, maintaining the availability of the security functions. Moreover, SBLWT does not require any trust in the normal OS scheduler or software interrupts for context switching, closing off an avenue where the OS could attempt to tamper with or delay the secure execution. Once in Secure World, the OS cannot intervene until the secure operation is finished, ensuring atomicity and integrity of those operations.

3.2.2 Limitations. Despite its strong security posture, SBLWT comes with several limitations and practical challenges:

- **Bitcoin-Only Design:** SBLWT is specifically tailored for Bitcoin (and Bitcoin-like) blockchains using SPV. The entire architecture (block headers, Merkle trees, Bloom filters) is based on Bitcoin's protocol. It does not support other cryptocurrency networks (e.g., Ethereum or others that don't use Bitcoin's blockchain structure) without significant redesign. There's no mechanism for multi-currency support; users dealing with multiple coins would need separate instances or entirely different solutions. This single-coin focus was common at the time, but it limits the wallet's applicability as the ecosystem diversifies.

- **No UI Trust Guarantees:** Like TBW, SBLWT does not solve the trusted user interface problem. The secure wallet has no dedicated display or input outside the normal OS. A malicious Android OS could still attempt phishing attacks such as intercepting a Bitcoin address the user copies and replacing it with an attacker's address. In fact, the paper explicitly notes that an attacker can tamper with the address shown to the user or modify the clipboard, potentially redirecting funds. SBLWT's countermeasure to address tampering is limited to generating and storing the wallet's own addresses securely, but when it comes to displaying information to the user or accepting destination addresses, the system ultimately relies on the Normal World. Without secure UI support (which typically requires special TrustZone frameworks and trusted display hardware), SBLWT cannot prevent a compromised OS from lying to the user. In short, human verification of transaction details is not protected – an adversary with control of the phone's UI could trick the user, which remains an open vulnerability.
- **Deployment Complexity:** The SBLWT model assumes a level of control over the device that may not be practical for end-users. It requires a custom secure bootloader and a custom TrustZone secure OS that launches before the normal OS. Implementing this on commercial smartphones would require cooperation from device manufacturers or flashing custom firmware. In 2018, and even today, most phone vendors do not allow third-party secure OSes to be installed. Thus, SBLWT was demonstrated on a development board (Raspberry Pi 3) rather than an actual phone, highlighting that this is a proof-of-concept. For real-world adoption, significant engineering would be needed to integrate SBLWT's approach into existing mobile platforms (which have their own TEEs and boot chains). This makes the solution less immediately applicable outside of laboratory settings.

3.3 Secure User-friendly Blockchain Modular Wallet Design Using Android OP-TEE (Kim et al., 2025)

3.3.1 Architecture. We proposed a modular blockchain wallet architecture that extends TrustZone-based wallets to support multiple cryptocurrencies and flexible updates[61]. This design, implemented on Android with OP-TEE, addresses limitations of earlier single-coin wallets by allowing dynamic loading of different blockchain-specific wallet modules into the TEE on demand. The architecture is composed of the following main components:

- **Android Client Application (Rich OS Component):** In the Normal World, a user-facing Android app provides a unified interface (UI) for managing multiple types of cryptocurrencies (e.g., BTC, ETH, Solana, etc.). This single app accommodates all supported coins so that users do not need separate wallet apps for each network. The app is responsible for general tasks like network communication with blockchain nodes/APIs, user inputs, displaying balances and transaction history, and triggering operations (like “create new wallet” or “send transaction”). Crucially, the client app is designed to be modular: users can choose which blockchain modules to install or activate through the UI, keeping the app lightweight by including only the desired coins.
- **OP-TEE Trusted Execution Environment:** On the Secure World side, OP-TEE is used as the TEE OS. OP-TEE was chosen for its open-source nature and compliance with GlobalPlatform standards, ensuring portability across devices and easier development of TAs. The secure wallet design consists of multiple Trusted Applications (TAs), each encapsulating the logic for a specific blockchain network's wallet operations. For example, there might be a Bitcoin TA, an Ethereum TA, a Solana TA, etc., each responsible for that network's key management, transaction signing, and any coin-specific cryptographic algorithms. OP-TEE's Multi-TA support allows these modules to be isolated from each other in separate secure world address spaces. The TAs communicate with the Normal World app via the OP-TEE client API, but they do not interact directly with one another (isolation helps contain any faults or vulnerabilities to one module). Initially, only a minimal set of TAs may be present, and others can be added via updates.
- **Dynamic Module Loading (OTA Updates):** A standout feature of this architecture is its ability to dynamically deploy and update wallet modules. The system uses a form of over-the-air (OTA) update mechanism to deliver new TAs to the device securely. The workflow is as follows: (1) A developer or the wallet vendor creates a new TA module for a blockchain (for instance, a module for “FooCoin”). The module is signed with a developer certificate or the vendor's private key and uploaded to an update server. (2) When a user wants to add this new coin in their wallet app, the Android client contacts the server and downloads the TA package (the OTA payload). (3) The client passes the encrypted/signed TA blob to the OP-TEE secure OS. OP-TEE then verifies the authenticity and integrity of the TA using a digital signature (e.g., RSA-PSS with SHA-256) and checks the hash. Only if the signature is valid (matching a trusted root or developer key that the TEE recognizes) will it load the TA into Secure World memory. (4) The new TA is installed in an isolated secure partition and becomes available for use. This process is akin to “installing an app” into the TEE, but controlled and verified by the TEE for security. By designing it this way, adding support for a new blockchain does not require modifying the core wallet app or performing a firmware flash of the entire device; it's as straightforward as a software update delivered through the app itself.
- **Modular TA Structure:** Each blockchain TA implements the necessary functions for that network. Typically, this includes key generation (using the appropriate elliptic curves or algorithms, e.g., secp256k1 for Bitcoin/Ethereum, Ed25519 for Solana), secure key storage (via OP-TEE's secure storage), address derivation (coin-specific address formats), and transaction signing. The TAs might also handle coin-specific transaction serialization or basic validation. For example, the Bitcoin TA could implement an SPV verification similar to SBLWT, or it might simply sign Bitcoin transactions if the design offloads verification to the server or app. The Ethereum TA might handle ECDSA signing and perhaps chainID, etc., leaving transaction assembly to the Normal World.

This separation means each TA is relatively small and focused, which improves security (smaller attack surface per TA) and maintainability. If a vulnerability is found in, say, the Bitcoin TA, it can be patched independently and updated without disturbing other modules.

- **Manager/Controller TA:** The paper describes dynamic composition on the server side, but on the device side, there may be a need for a central TA or secure service that manages the lifecycle of other TAs (loading/unloading) and possibly coordinates between them and the Normal World. It's not explicitly named in the snippet, but we can infer that OP-TEE's normal operation would treat the client app as calling into different TAs via UUIDs. The client app must know which TA to call for which coin's operations. This mapping could be handled by the Normal World logic (it knows "use TA X for Bitcoin, TA Y for Ethereum"). The architecture ensures that from the user's perspective, this is seamless: the user just uses the unified UI, and under the hood the correct TA is invoked for the coin they are transacting with.

3.3.2 Limitations. The modular wallet design is forward-looking, but it also has certain limitations and challenges:

- **No TrustZone UI for Confirmation:** The architecture still relies on the normal Android UI for user interactions. It does not inherently solve the problem of a compromised OS deceiving the user, similar to the previous two solutions. If the Android OS is malware-infected, it could, for example, display a fake recipient address or amount to the user while sending different data to the TA for signing. Without a hardware-isolated display or input (such as a Trusted UI where the TEE draws the prompt), the user must trust that the app is showing correct information. This is a known open issue – the design improves back-end security but cannot guarantee the user isn't tricked via a fraudulent interface. The authors did not indicate a custom secure UI implementation (which would likely require OEM support), so this limitation remains: an attacker with complete control of the OS could attempt phishing attacks or input capture (e.g., intercepting PIN entry). In mitigation, the system could integrate with Android's accessibility or security features (like using Android's confirmation dialog that leverages TEE in newer versions), but such integration is not described and would depend on device support.
- **Complexity of Update Infrastructure:** While OTA updates of TAs bring flexibility, they introduce a complex supply chain. A secure distribution server is needed, along with a robust process for module signing, versioning, and possibly review (especially if third-party developers can submit modules). This raises questions: Who signs the modules? Is there a central authority or could there be multiple trusted publishers? If community developers contribute (as hinted by the mention of developer submission), there must be a vetting process to avoid malicious modules. This governance aspect is a limitation in that it adds operational overhead and potential single points of failure or trust. If the update server is compromised or the signing keys are stolen, an attacker could distribute a malicious TA. The architecture's security then hinges on protecting the update pipeline. The design addresses this by requiring TA signature verification

on-device, but one can imagine social engineering or Denial-of-Service attacks on the update system as new threat vectors. In summary, the flexibility introduces a new layer (the update server and process) that itself must be secured and maintained – something static wallets didn't have to worry about.

4 Architecture

4.1 Overview

MANDARIN is a secure and user-friendly blockchain modular wallet OS that aims to reconcile two persistent tensions in real-world wallet deployments: **strong private-key protection** under realistic adversaries, and **low-friction usability** for everyday users who increasingly interact with multiple blockchain networks. The system is motivated by the observation that, in practice, the private key (or any signing authority that can derive valid signatures) is the single most security-critical asset in a wallet, and therefore it should be protected by a hardware-enforced isolation boundary rather than being stored and used directly inside a large commodity OS runtime.

To achieve this, MANDARIN adopts a TEE-centered architecture based on **Arm TrustZone** and **OP-TEE**, while using **Android** as the user-facing platform. TrustZone provides the architectural split between a Secure world and a Normal world, allowing sensitive operations to execute in a security state that is isolated from Android's application and service ecosystem [62]. OP-TEE supplies the secure execution environment in which **Trusted Applications (TAs)** can run with isolation guarantees and where TA loading can be conditioned on cryptographic verification, even when TA binaries are stored in an untrusted filesystem [63]. In MANDARIN, this yields a clear division of responsibility: Android (REE) focuses on user experience and module lifecycle management, while OP-TEE (TEE) becomes the root environment for key custody and signing.

A second core motivation for MANDARIN is the usability bottleneck created by today's multi-network ecosystem. Because blockchain networks differ in signature schemes, transaction structures, and feature sets (e.g., token/NFT semantics in some ecosystems), users commonly end up installing multiple wallet applications to cover the networks they care about. This fragments user experience, increases operational overhead, and often multiplies security exposure through duplicated codebases and inconsistent update practices. MANDARIN addresses this problem by treating network support as an extensibility concern rather than a property of a monolithic wallet application: users install network functionality as **modules**, while a single master wallet application coordinates and manages those modules.

Concretely, a MANDARIN module packages three complementary artifacts: (i) **MINIAPP**, (ii) **CA**, and (iii) **TA**. **MINIAPP** is a lightweight user-facing component implemented with **HTML + CSS + JavaScript** and is loaded from **local app resources** (rather than remote content), allowing the platform to provide a consistent UI shell while enabling network-specific workflows to be expressed flexibly. The module's **CA (Client Application)** is a network-specific native binary that implements the concrete set of functions

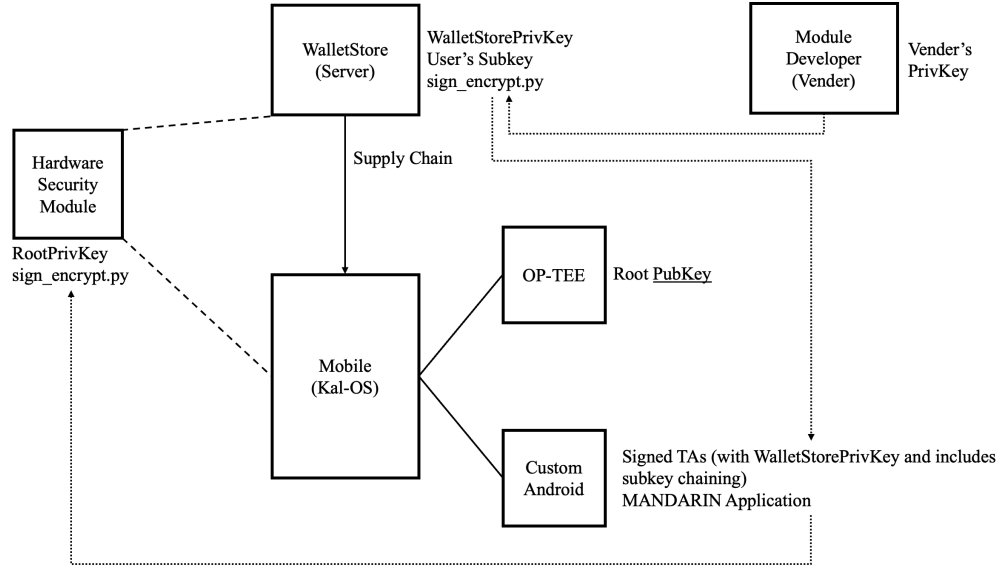


Figure 3: Architecture of MANDARIN

required by that network (including potentially richer feature surfaces for certain ecosystems). The module’s **TA (Trusted Application)** encapsulates security-sensitive operations, most importantly signing, and executes inside OP-TEE under TEE isolation.

MANDARIN defines a straightforward invocation path from UI interaction to secure signing. MINIAPP runs in an Android WebView context and interacts with the Android application layer through a controlled native bridge mechanism; the request is then routed through MANDARIN’s managed layer and finally into native code via JNI [64]. The module CA is invoked from this native layer and communicates with the TA through OP-TEE’s client/server model, which follows the GlobalPlatform **TEE Client API** semantics (identify TA by UUID, open a session, invoke a command) [65, 66]. At the hardware boundary, the call is realized through a TrustZone world switch (commonly mediated through **SMC** transitions), after which the TA executes the requested operation in Secure world [62, 63].

Because modules are installed dynamically, MANDARIN must ensure that only authorized secure components can execute in OP-TEE. For this reason, MANDARIN requires that module TAs are delivered as **Subkey-signed** artifacts that can be verified by OP-TEE at load time. OP-TEE Subkeys provide a public-key hierarchy that supports delegated TA signing, and they impose **UUIDv5 namespace constraints** to prevent accidental or adversarial TA UUID collisions across different signing domains [67]. MANDARIN also uses **UUIDv5** as a deterministic identifier mechanism to bind CA and TA identities and to maintain stable module naming across the lifecycle of installation, invocation, and update [68]. This design makes module management tractable at scale while strengthening the security story: the REE may store module artifacts, but the TEE admits execution only after cryptographic validation.

From the user’s perspective, the outcome is a master wallet experience where supporting “N networks” does not require installing “N wallet apps.” Instead, users install the modules they need, and

MANDARIN manages them under a single application boundary. This architecture also creates a practical distribution channel for smaller blockchain ecosystems: rather than persuading users to install an entirely new wallet, a network vendor can publish a module that plugs into an existing secure wallet platform, reducing adoption friction while still benefiting from OP-TEE-backed key isolation.

Overall, the MANDARIN overview can be summarized as follows: **Android** provides usability and module orchestration, **OP-TEE** provides isolated execution for signing and key protection, and a **modular packaging model (MINIAPP + CA + TA)** allows the system to scale across heterogeneous blockchain networks without forcing users into fragmented wallet application installations. The remainder of the paper details these components and their interactions—MANDARIN core, the Module lifecycle, and the WalletStore server—along with the security and operational rationale behind each design choice.

4.2 OP-TEE

4.2.1 Why OP-TEE: a TrustZone-based TEE for Android-class Arm systems. MANDARIN targets a deployment environment in which the dominant CPU architecture is **Arm** and the dominant user-facing OS is **Android**. This choice is driven by deployment realism: a “secure and user-friendly” wallet platform must integrate with mature mobile UI/UX ecosystems, application frameworks, hardware abstraction layers, and device I/O stacks that Android provides at scale. At the same time, Android’s breadth—system services, OEM customizations, third-party applications, network-facing components, and continuous update churn—implies an expansive and evolving attack surface. In response, MANDARIN adopts a principled separation in which **private-key custody and signing authority are confined to a hardware-isolated domain**, while

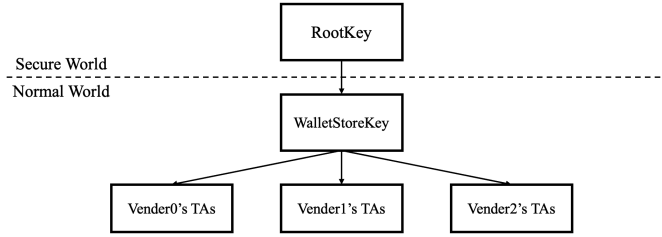


Figure 4: Subkey Hierarchy

Android remains responsible for usability, orchestration, networking, and module presentation.

To implement this separation, MANDARIN relies on **Arm TrustZone**, which partitions execution into Secure and Non-secure security states and enables a Trusted Execution Environment (TEE) to execute in isolation from the commodity OS [69]. TrustZone is particularly well-suited for wallet architectures because it supports a clean and reviewable trust boundary: secrets can remain resident in secure memory, and the Rich Execution Environment (REE) can be treated as untrusted with respect to both key confidentiality and signing correctness. In MANDARIN, this boundary is used to ensure that any cryptographic operation that confers authorization (e.g., producing valid transaction signatures) occurs inside the secure domain, regardless of how untrusted the Android-side runtime might become.

On top of TrustZone, MANDARIN uses **OP-TEE** as the TEE OS. OP-TEE provides a mature runtime organized around **Trusted Applications (TAs)** and a well-defined client interaction model grounded in GlobalPlatform abstractions [70, 71]. In particular, OP-TEE follows a session-based CA-TA communication pattern: a **Client Application (CA)** in the REE identifies the TA by UUID, opens a session, and invokes constrained commands using a commandID [71, 72]. This model is valuable for MANDARIN because it naturally supports compartmentalization: network-specific signing logic can be encapsulated into individual TAs, while the Android-side system remains a unified “master wallet” that routes requests to the appropriate secure component.

4.2.2 Practical integration: STM32MP257F-EV1 + OpenSTDRoid as a reproducible baseline. Although the TrustZone + TEE architecture is conceptually straightforward, building a complete **Android + OP-TEE** stack on embedded hardware is frequently challenging in practice. A working system requires consistent alignment across the boot chain, secure monitor handoff, firmware packaging, Linux kernel drivers (including OP-TEE drivers), user-space components such as tee-supplciant, partition layout, and board-specific BSP details. When these platform details are unstable, they can dominate engineering effort and obscure the architectural contribution of the wallet OS itself.

To avoid this pitfall while still targeting a representative Arm + Android environment, MANDARIN selects **STM32MP257F-EV1** as the reference platform and adopts **OpenSTDRoid** as the Android distribution baseline. ST’s documentation positions OpenSTDRoid as an Android delivery validated against the STM32 MPU ecosystem release v5.1.0 and provides a set of guides (including TEE-related build guidance) that stabilize the platform bring-up process [73–76].

In effect, STM32MP257F-EV1 + OpenSTDRoid serves as an integration substrate that reduces non-essential variability, enabling the paper to focus on the core research problem: **how to securely support dynamic, network-specific wallet modules on Android while keeping signing authority inside OP-TEE.**

It is also notable that STM32MP25-class devices explicitly advertise TrustZone support for secure operations, which aligns with the design intent of using TrustZone-backed isolation as the root security primitive [77].

4.2.3 Dynamic TA delivery: aligning modular wallets with OP-TEE’s TA lifecycle. A core requirement of MANDARIN is **dynamic TA support**. Because blockchain ecosystems differ in signature schemes, transaction layouts, and validation rules, a practical multi-network wallet cannot scale if all logic must be embedded into a single TA or a single wallet binary. Instead, MANDARIN treats network support as modular logic that must be installable and updateable post-deployment. This requirement directly implies that TAs corresponding to network modules must be distributable after device shipment.

OP-TEE’s TA model supports deployable TA artifacts that can reside in the REE filesystem and be loaded on demand. OP-TEE documentation describes that user-mode TAs are ELF-based artifacts named after the TA UUID with a `.ta` suffix, and are *signed* (and optionally *encrypted*) as part of the TA packaging process [78]. This is important for MANDARIN: it enables the system to store module TAs in the REE (where installation and updates are operationally convenient), while relying on **Secure world verification at load time** to enforce authenticity and integrity before any TA code is admitted for execution.

MANDARIN intentionally adopts a **sign-only** TA packaging posture, rather than encrypting the TA payload. This is a deliberate design decision: the primary requirement is to prevent adversaries in the REE from injecting or modifying trusted signing logic, not necessarily to keep TA code confidential. In this model, confidentiality can be addressed at other layers if required, but the security-critical guarantee is that only authorized, correctly signed TAs can execute inside OP-TEE.

4.2.4 Why Subkeys: delegated signing authority for a modular TA ecosystem. Dynamic TA distribution raises an immediate governance and key-management challenge. If all TAs were signed under a single global private key, then compromise of that key would be catastrophic, and it would be difficult to decentralize module publishing or operational signing responsibilities across stakeholders. This challenge is amplified in a modular wallet OS, where the

number of supported networks (and thus the number of TAs) can grow over time, and where module updates may occur frequently.

To address this, MANDARIN adopts OP-TEE’s **Subkeys** mechanism. Subkeys provide an OP-TEE-specific implementation of a **public-key hierarchy** for TA signing, enabling delegated signing authority without distributing a single omnipotent private key [79]. In the Subkeys model, a root key verifies the first-level subkeys, which verify second-level subkeys, and so on. OP-TEE further constrains Subkey-based signing by confining TAs signed with a subkey to the **UUIDv5 namespace** defined by that subkey, which reduces the risk of TA UUID clashes across different signing domains [78, 79]. This namespace property is consistent with UUIDv5 semantics as standardized in RFC 4122 [80] and is operationally useful in modular systems because it links TA identity management to the authorization chain.

From MANDARIN’s perspective, Subkeys are not merely an optional enhancement; they are the enabling mechanism that makes “dynamic TA modules” operationally realistic while preserving a strong chain-of-trust narrative.

4.2.5 MANDARIN Subkey hierarchy: device-unique root anchoring and WalletStore delegation. MANDARIN defines a Subkey hierarchy that binds TA authorization to a device-specific trust anchor and delegates signing authority to the WalletStore tier. The high-level structure is:

- (1) **RootKey** (device-unique trust anchor; root public key embedded per device in optee_os at build time)
- (2) **WalletStoreSubKey** (WalletStorePubKey signed by RootPrivKey; represented operationally as an .sk artifact)
- (3) **TAIntegrityCheckSubkey / TA signature binding** (TA binary digest signed by WalletStorePrivKey, with WalletStoreSubKey attached to form a verifiable chain)

This hierarchy embodies two design intents.

Device-unique trust anchoring. In MANDARIN, the trust anchor is *device-scoped*. Concretely, we embed a device-specific RootPubKey into the optee_os build (per-device embedding). The operational rationale is containment: a TA artifact that verifies under one device’s root domain is not automatically valid for another device. This property reduces blast radius under credential compromise and naturally supports per-device authorization policies.

WalletStore delegation without root key exposure. The second level, WalletStoreSubKey, is generated by signing WalletStorePubKey using the device’s RootPrivKey, producing a Subkey artifact (.sk) that the WalletStore server can later attach to module deliveries. This step establishes a delegation chain from the device root anchor to a WalletStore-controlled key domain without requiring the WalletStore tier to possess RootPrivKey. Subkeys are explicitly designed as a hierarchical mechanism where each hop can be verified step-by-step from the root [79].

At the TA level, each TA package is bound to integrity by signing the TA binary digest using **WalletStorePrivKey** (managed inside a server-side HSM; see Section 4.2.6). Crucially, MANDARIN requires that the TA delivery includes the previously generated

WalletStoreSubKey, so that the verifier can reconstruct the chain:

```
RootPubKey (embedded) → WalletStoreSubKey
                          → TA integrity signature.
```

This “attach the previous subkey” rule is what makes the trust chain self-contained from the verifier’s perspective. OP-TEE describes subkey verification as a chain rooted in a known root key and notes that TAs can be verified with a subkey or a chain of subkeys [78].

4.2.6 Server-side TA finalization: returning a complete TA artifact at download time. MANDARIN imposes a strict constraint on downloadable modules: **the TA must be complete at download time**. A “complete” TA here means that the TA artifact already includes the required Subkey chain elements (except the embedded root anchor) and can be verified immediately by OP-TEE’s TA loader. This constraint is threat-driven: Android (REE) is not trusted to “finish” signing or authorization steps because malware could interfere with admission control. Instead, the secure world must make the final admission decision based solely on cryptographic verification over the delivered artifact.

Therefore, MANDARIN performs **final TA signing on the server side**. The server must possess (or have secure access to) the credentials required to produce a TA that validates under the device’s trust domain. MANDARIN implements this as follows:

Provisioning phase (pre-deployment). Prior to device distribution, an HSM-backed provisioning process signs WalletStorePubKey using the device’s RootPrivKey to generate the device-specific WalletStoreSubKey (.sk). The resulting .sk artifact is stored in the WalletStore database indexed by device identity. This minimizes online exposure of RootPrivKey; the root key is used to establish delegation and does not need to remain online for routine module delivery.

Delivery phase (runtime download). When the device requests a TA download, the server retrieves the corresponding device-specific WalletStoreSubKey from the database, computes the TA digest, and produces the TA integrity signature using WalletStorePrivKey, which is held in a server-side HSM. The server then assembles a final, verifiable TA artifact that includes both (i) the integrity signature binding and (ii) the attached WalletStoreSubKey. The device thus receives a TA that OP-TEE can verify at load time without any local signing steps.

This workflow aligns with OP-TEE’s TA build and signing model, which explicitly describes producing a signed TA artifact using the OP-TEE toolchain and notes that `sign_encrypt.py` attaches signatures to form deployable .ta artifacts [81, 82].

4.2.7 Tooling: implementing Subkey generation and TA packaging via sign_encrypt.py (sign-only). MANDARIN standardizes Subkey generation, TA signing, and final TA assembly using OP-TEE’s `sign_encrypt.py`. OP-TEE documentation states that TAs are signed using `sign_encrypt.py` and that its default behavior is to sign a compiled TA binary and attach the signature to form a complete TA for deployment [81, 82]. The script is also published as part of the optee_os repository [83].

In MANDARIN, we use the signing pathway without encryption. This is consistent with our threat model focus: we require strong integrity/authenticity guarantees for trusted signing logic,

while confidentiality of TA code is not a primary requirement in the baseline design. Operationally, using `sign_encrypt.py` across both developer-side build workflows and server-side delivery workflows reduces accidental divergence: the same artifact format and verification assumptions are preserved end-to-end.

4.2.8 Implications for a modular wallet OS. The OP-TEE + Subkeys design yields several system-level benefits for MANDARIN:

Scalable trust delegation. Subkeys enable MANDARIN to avoid a single global signing key for all modules, supporting modular growth and operational delegation while retaining a verifiable chain of authorization [79, 84, 85].

Device-scoped authorization. Per-device root embedding supports device-specific authorization by construction. In large deployments, this can reduce the impact of compromised delegated credentials and can support per-device policy constraints.

TEE-enforced admission control over untrusted storage. Because OP-TEE verifies TA artifacts at load time (even when stored in the REE filesystem), the REE can be treated as an untrusted transport and storage plane, while the secure world remains the policy enforcement point for TA admission [78].

HSM-aligned server-side signing. By managing WalletStorePrivKey inside an HSM and performing server-side finalization, MANDARIN aligns TA distribution with common key-management best practices: highly sensitive signing keys are not required to exist in software-accessible memory in online services.

In summary, OP-TEE is not merely “a secure place to store keys” in MANDARIN. It is the secure execution substrate that enables a modular wallet OS to safely support dynamic, network-specific signing logic, while Subkeys provide the delegated key hierarchy needed for scalable, device-scoped TA authorization.

4.3 MANDARIN

MANDARIN is a **system application** deployed on a customized Android distribution and serves as the *master wallet runtime* of our modular wallet OS. Rather than embedding every blockchain network into a single monolithic wallet application, MANDARIN provides a stable, policy-driven *platform layer* that hosts a family of installable network modules. In this architecture, MANDARIN is responsible for consistent onboarding, lifecycle management, and secure invocation orchestration, while network-specific behavior is encapsulated inside modules.

The design of MANDARIN is shaped by two practical observations. First, modern mobile wallets operate in a threat environment where the Rich Execution Environment (REE) is large, complex, and continuously exposed to application-layer attack surfaces. Second, multi-chain usage is no longer exceptional; users increasingly interact with heterogeneous networks whose transaction formats, signing rules, and feature surfaces diverge significantly over time. MANDARIN addresses these challenges by treating Android as the usability and orchestration plane and OP-TEE as the authority for key isolation and signing execution, while introducing a modular packaging model that makes “adding a network” an installation event rather than an app-installation burden.

From a deployment perspective, MANDARIN is implemented as a **system app**. This is not merely a packaging detail: modern Android platforms place strict constraints on privileged behaviors, requiring that privileged permissions be explicitly allowlisted. Android system documentation further notes that on Android 9 and higher, privileged permissions must be allowlisted (otherwise the device may fail to boot or privileged permissions may not be granted), motivating deterministic platform integration for a master wallet runtime that must reliably manage module artifacts and enforce consistent policy across devices [86, 87].

4.3.1 Architectural role and responsibilities. MANDARIN provides three core capabilities.

(1) *WalletStore-driven module acquisition and installation.* MANDARIN renders a WalletStore web service in a WebView context so that users can browse and install network modules. A module payload includes **MINIAPP**, **CA**, and **TA**. MANDARIN stores each artifact into an appropriate location for its runtime semantics, with particular attention to TA placement: the TA must be placed in the **OP-TEE REE filesystem TA store** so that OP-TEE can locate and load it by its UUID through the standard REE filesystem loading mechanism [88].

(2) *End-to-end invocation orchestration:* $\text{MINIAPP} \rightarrow \text{MANDARIN} \rightarrow \text{JNI} \rightarrow \text{CA} \rightarrow \text{TA}$. MANDARIN provides the canonical pathway for module execution and secure signing requests. MINIAPP and Java/Kotlin code do not directly invoke a TA. Instead, MANDARIN uses **JNI** to cross into native code, where the module-specific CA performs OP-TEE client operations that identify the TA by UUID and invoke commands by `commandID` [89, 90].

(3) *Module lifecycle management and execution.* MANDARIN maintains the installed module inventory, allows the user to select a module, launches the corresponding MINIAPP, and manages module updates and deletion. Critically, MANDARIN enforces **atomic update** semantics to prevent partially-updated module states across the MINIAPP/CA/TA triplet.

Taken together, these responsibilities position MANDARIN as a modular wallet “OS service” at the application layer: it standardizes distribution, execution, and routing policy, while delegating cryptographic authority to the TEE and delegating network-specific semantics to modules.

4.3.2 WalletStore integration in WebView. MANDARIN integrates WalletStore using **WebView** to provide a familiar store-like onboarding experience while preserving platform control over installation semantics. Since the primary interaction is “discover $\rightarrow$ download $\rightarrow$ install,” WebView becomes a natural container for store UI and metadata presentation. However, WebView also implies security obligations: once web content is allowed to interact with native functionality (e.g., via a JavaScript bridge), the platform must clearly define what content is trusted and how bridge access is constrained.

For WalletStore communication, MANDARIN assumes HTTPS and adopts **certificate pinning** to strengthen server authenticity against MitM-level attackers. Android documents certificate pinning in Network Security Configuration as pinning hashes of certificate public keys (the `SubjectPublicKeyInfo`), and requiring

that a validated chain include at least one pinned public key [91]. Android also cautions that pinning is operationally brittle (e.g., server certificate or CA rotations can break connectivity without coordinated client updates), motivating disciplined pinset lifecycle management [92].

A useful conceptual distinction in MANDARIN is that WalletStore is only the *distribution entry point*, not the runtime of MINIAPP. WalletStore pages are used for discovery and installation, while MINIAPP execution occurs from local resources after installation (subsection 4.3.4). This split reduces reliance on remote content during normal wallet operations and makes it easier to reason about the trust boundary of the WebView bridge.

4.3.3 Module installation: artifact classification and placement policy. A module contains three artifacts:

- **MINIAPP:** a web-based user interface implemented in HTML/CSS/JavaScript.
- **CA:** a per-module native client component that performs network-specific native operations and TEE invocation.
- **TA:** a trusted component executed within OP-TEE.

MANDARIN’s installer classifies artifacts and stores them accordingly. The most critical placement rule is for TAs: the TA must be stored in the **OP-TEE REE filesystem TA store**, enabling dynamic provisioning (install/update/delete) while relying on OP-TEE load-time verification to admit execution [88]. The OP-TEE client/server model identifies the TA by UUID and dispatches requests via `commandID`, which naturally maps to a modular wallet where each TA exposes a small set of stable secure commands [72, 90].

CA placement is intentionally treated as an implementation detail in the paper, because the key architectural point is that CA is **per-module** and provides a network-specific native surface. This is important in practice because different networks may require distinct function families. Even within a single ecosystem (e.g., Ethereum-class networks), required features can extend beyond base transfers into token/NFT operations and contract interaction flows. By isolating CA per module, MANDARIN avoids turning the master wallet into a monolithic native codebase and avoids imposing a single ABI that must anticipate all future network behaviors.

Finally, MINIAPP is installed as **local app resources** and is executed locally after installation. This is compatible with a threat-informed stance: MINIAPP can conduct network I/O, but it must not hold signing authority; instead, it prepares unsigned data and requests signatures through the platform interface.

4.3.4 MINIAPP execution model: local resources with a hardened origin. MINIAPP is loaded primarily from assets/resources. MANDARIN can strengthen this design by using **WebViewAssetLoader**, which Android describes as a flexible mechanism to load in-app content using HTTP(S)-style URLs, improving compatibility with the same-origin policy and supporting subresources such as JavaScript, CSS, images, and iframes [93, 94].

This hardening step is valuable for three reasons:

- (1) **Origin clarity and policy leverage.** A stable HTTPS-like origin makes it easier to apply origin-based restrictions and reason about WebView security properties.

- (2) **Bridge containment.** MANDARIN’s WebView must expose a small set of privileged interfaces (e.g., “request signature”). Android explicitly warns that insecure native bridges (e.g., those implemented via `addJavaScriptInterface`) can introduce risks such as interface injection and XSS-driven abuse if untrusted content becomes reachable [95].
- (3) **Best-practice alignment.** Using `WebViewAssetLoader` for local content aligns with widely adopted guidance for safer local-content loading in WebView environments [96].

Accordingly, MINIAPP remains local and packaged, but is delivered under a controlled origin that enables stricter bridge policies and more predictable web security behavior.

4.3.5 WebView bridge and request mediation. MANDARIN’s WebView bridge is the interface between MINIAPP logic and the Android application layer. Because MANDARIN runs a **single WebView runtime** for all modules, the bridge must be designed as a shared capability that is carefully scoped to avoid cross-module confusion or accidental privilege exposure.

Security guidance emphasizes that bridges should be minimal and constrained. Android warns that native bridges can lead to interface injection and unintended manipulation if untrusted content is loaded [95]. OWASP guidance similarly highlights that if a bridge is necessary, only trusted (packaged) JavaScript should access it, and the exposed surface should be restricted [96].

In MANDARIN, the bridge is treated as a **request gateway** rather than a general-purpose native API. MINIAPP does not receive raw access to TEE semantics; instead, it emits structured requests (e.g., “sign this preimage,” “derive address for path,” “query module metadata”), and MANDARIN performs request mediation before forwarding to native layers. This mediation is also the natural place to implement platform policy such as argument validation, module binding checks, rate limiting, and optional user-consent enforcement.

4.3.6 JNI gateway and CA execution. MANDARIN uses **JNI** to transition from Java/Kotlin managed code into native code for CA invocation. Android’s JNI guidance describes JNI as a vendor-neutral interface that allows managed code to interact with native code loaded from dynamic shared libraries [89].

In MANDARIN, JNI is more than a technical necessity: it is an architectural boundary that supports modularity and auditability. Concretely:

- JNI allows MANDARIN to maintain a stable managed-code platform layer while supporting per-module native implementations.
- JNI provides a choke point where MANDARIN can enforce module selection context (ensuring calls from a given MINIAPP are routed to the corresponding CA).
- JNI separates UI behavior (WebView) from privilege-bearing functionality (native code and TEE invocation).

The CA, as a per-module native component, then performs any network-specific native logic and serves as the client-side endpoint for OP-TEE invocation.

4.3.7 CA ↔ TA communication: UUID addressing and command dispatch. The CA invokes the TA following the `GlobalPlatform`-style client/server model: to identify a TA, the client provides a

UUID, and to select a function, the client provides a `commandID` [90]. GlobalPlatform’s TEE Client API specification likewise defines that sessions are created by identifying the TA using its UUID, aligning with a modular wallet where each secure module is bound to a stable secure identifier [72].

This model benefits modular wallets because it cleanly supports evolution. A TA can expose multiple commands (e.g., sign, derive, validate) without changing the identification scheme. Meanwhile, MANDARIN can remain agnostic to specific cryptographic algorithms and transaction structures: it simply routes requests to the correct CA/TA pair according to the currently selected module context.

4.3.8 Network communication in MINIAPP: online UI, offline authority. MANDARIN enforces a clear separation: MINIAPP primarily handles UI and network interaction, while **signing must be executed in TA**. This separation can be framed as *online UI, offline authority*. In this model, MINIAPP is responsible for:

- fetching chain state (balances, UTXO sets, account nonces, gas/fee data, token metadata);
- preparing an **unsigned** transaction payload and presenting it to the user;
- broadcasting signed transactions to the network after receiving a signature.

In contrast, the TA is responsible for:

- holding or deriving signing keys (as defined in the key-management design);
- signing the exact preimages specified by the module protocol;
- optionally enforcing module-level policy checks (e.g., key usage constraints, derivation path restrictions, or domain separation).

This separation yields two key benefits. First, it reduces secret exposure in the networked layer: MINIAPP performs network I/O, but signing authority never leaves the TEE. Second, it provides a uniform security posture across heterogeneous networks: regardless of network semantics, actions that authorize value transfer must cross a secure boundary. However, because MINIAPP performs network I/O, MANDARIN must constrain WebView navigation and content loading so that untrusted remote pages cannot gain access to privileged bridges. A hardened design permits remote communication through `fetch/XHR/WebSocket` while restricting top-level navigation away from trusted local origins.

4.3.9 Lifecycle management: install, update, delete. MANDARIN manages the full lifecycle of modules: installation, update, and deletion. This is particularly important in a modular wallet because correctness and security depend on **coherence across multiple artifacts**.

Atomic update semantics

MANDARIN adopts **atomic update** semantics: updates must not leave the system in an intermediate state where MINIAPP, CA, and TA versions mismatch. A robust update procedure can be described as a transactional workflow:

- (1) **Stage** the downloaded artifacts into a temporary location.

- (2) **Validate** completeness (MINIAPP/CA/TA all present) and consistency (manifest metadata, version constraints, module identifiers).
- (3) **Commit atomically** by updating a registry pointer or renaming the staged directory into place using an atomic filesystem operation.
- (4) **Rollback on failure** by discarding the staged artifacts and leaving the previously installed module untouched.

Version matching across MINIAPP/CA/TA can be enforced via module metadata to prevent UI/native/TEE interface mismatches after upgrade.

Deletion semantics and state hygiene

Module deletion is treated as more than removing a UI entry. MANDARIN deactivates the module in its registry (preventing further invocation), removes artifacts (including removing the TA from the REE filesystem store), and clears module-scoped WebView state if applicable (e.g., `localStorage/IndexedDB/cache` associated with the module origin). This prevents stale UI state or cached responses from persisting across uninstall/reinstall cycles, especially relevant given the single WebView runtime design.

4.3.10 Version rollback considerations (platform + TEE). Dynamic module systems face an additional risk: an attacker with write access to untrusted storage may attempt to replace a module TA with an older, vulnerable version. OP-TEE documents a rollback protection mechanism for REE filesystem TAs using a secure `ta_ver.db` that tracks TA versions and can prevent downgrades, with effectiveness depending on secure storage configuration (e.g., RPMB-backed secure storage) [97].

MANDARIN’s atomic update addresses correctness in the REE (avoiding partial updates), while OP-TEE rollback protection addresses a deeper class of adversary behavior (forced downgrade). Together, they strengthen the modular wallet story: the platform installs and updates coherently, and the TEE can enforce monotonic secure component versions if configured.

4.3.11 Summary. MANDARIN is the component that makes the modular wallet architecture operational and usable. It provides:

- a WalletStore-backed onboarding workflow in WebView, secured by HTTPS and optionally certificate pinning [91];
- a hardened local MINIAPP execution model (`assets/resources`) that can be strengthened by `WebViewAssetLoader` to provide stable origin semantics and improved same-origin compatibility [93, 94];
- a controlled WebView bridge and request mediation layer informed by Android and OWASP guidance on native bridge risks [95, 96];
- a JNI gateway to invoke per-module CA logic, consistent with Android’s definition of JNI as the managed-to-native interface [89];
- a standardized CA↔TA invocation model grounded in UUID addressing and command dispatch, as described by OP-TEE and GlobalPlatform [72, 90];
- full lifecycle management (install/update/delete) with atomic update semantics, enabling coherent versioning across MINIAPP/CA/TA and aligning with OP-TEE’s optional TA rollback protection mechanisms [97].

This subsection describes MANDARIN as the platform runtime. Subsequent subsections specify the module format (identities, manifests, Subkey rules), the WalletStore server model, and TA/CA implementation details required to realize end-to-end security and usability goals.

4.4 Data Flow

MANDARIN implements a request/response data flow that preserves a uniform user experience while safely accommodating heterogeneous blockchain networks whose low-level execution semantics differ substantially. The workflow begins in the module’s **MINIAPP**, where the user supplies input values and triggers a desired operation. Although end users may perceive these operations as common wallet actions (e.g., *send*, *sign*, *approve*, *swap*), the concrete meaning of an operation is inherently network-specific: even when the user intent is similar, distinct blockchains can differ in transaction serialization, signing preimages, signature algorithms, domain separation rules, fee models, and extended feature surfaces. MANDARIN therefore treats every user-triggered action as a *module-scoped invocation* that must be interpreted under the currently active module context.

4.4.1 Unified invocation API: `CallTA(functionName, argsMap)`. To standardize how UI logic requests privileged operations, MANDARIN exposes a single invocation entry point, `CallTA`, which accepts two parameters:

- (1) `functionName`: a *string identifier* denoting the requested operation, and
- (2) `argsMap`: a dictionary-style argument map containing operation parameters.

Using a string-based function identifier simplifies module extensibility. The platform does not require compile-time enumeration of all possible module actions; instead, each module defines a vocabulary of function names that can evolve alongside its feature set. At the same time, the function-name abstraction allows MANDARIN to remain largely agnostic to protocol details: the platform routes the request based on the function identifier and module context, while network-specific semantics are handled downstream by module-specific native logic.

4.4.2 Java-to-native marshaling: structured Map propagation into JNI. The argument payload is constructed at the Android application layer and propagated to JNI as a structured Map, rather than as a raw serialized string. This is an important engineering decision because it reduces ambiguity at the boundary: JNI receives a structured representation that is naturally aligned with platform typing rules, enabling predictable extraction of strings, numbers, and (where applicable) byte arrays. As a result, JNI can apply common input handling policies (e.g., checking presence of mandatory keys, rejecting unexpected types, canonicalizing encodings) before module code is invoked. This approach also reduces reliance on ad-hoc parsing logic that would otherwise proliferate across modules.

4.4.3 JNI dispatch and deterministic module routing. After `CallTA` is issued, the request crosses into native code through JNI, where the platform performs parsing and dispatch. JNI interprets the `functionName` string and extracts parameters from `argsMap`, then

routes execution to the appropriate **CA** entry point. Crucially, dispatch is not solely a function-name lookup; it is a *module-scoped* dispatch determined by the currently active module.

MANDARIN enforces a strict binding rule: the invoked CA (and the downstream TA) must correspond to the **same UUIDv5 identity as the currently executing MINIAPP context**. This binding is enforced to prevent a malicious or compromised MINIAPP from invoking another module’s privileged functionality. Without such binding, an attacker could attempt cross-module privilege abuse—for example, triggering a signing procedure or key-related operation in a different module by crafting a `functionName` that exists there. To eliminate this class of confusion attacks, MANDARIN treats the module identity as a non-negotiable execution context: routing is derived from the active module’s UUIDv5, and the MINIAPP cannot arbitrarily select another module identity through API parameters.

Operationally, this creates a stable and auditable mapping:

$$\text{Active MINIAPP} \xrightarrow{\text{UUIDv5}} \text{Module CA} \xrightarrow{\text{UUIDv5}} \text{Module TA}.$$

Thus, even though `functionName` is a string and could theoretically name any operation, it is always interpreted within the namespace of the active module, and cannot escape into another module’s CA/TA domain.

4.4.4 Secure invocation: CA $\rightarrow$ TA via TrustZone transition. Once JNI dispatch selects the module CA, the CA performs any module-defined native processing and then invokes the corresponding TA. The invocation crosses the REE/TEE boundary using the platform’s standard secure-world call mechanism (commonly realized via an **SMC**-mediated world switch beneath the client API). The TA executes security-sensitive logic in the isolated OP-TEE environment, most notably signing operations. The key architectural consequence is that signing authority is never delegated to the REE: even when MINIAPP performs network communication and constructs unsigned transaction payloads, the act of producing an authorization signature remains confined to Secure world.

4.4.5 Asynchronous execution model for UI responsiveness. Because MANDARIN executes within an Android UI application, the data flow is implemented using an **asynchronous** invocation model. Calls to `CallTA` do not block the UI thread; instead, requests are dispatched to a background execution context and results are delivered back to the MINIAPP via asynchronous completion primitives (e.g., callback or promise-like completion). This is essential because secure-world calls may incur measurable latency due to context switching, TA loading, cryptographic computation, and IPC. Asynchronous invocation preserves interactive UX and prevents ANR behavior, while still enabling secure-world enforcement for signing and other privileged functions.

4.4.6 Return path and broadcast. After TA completion, the result is returned along the reverse path:

$$\text{TA} \rightarrow \text{CA} \rightarrow \text{JNI} \rightarrow \text{MANDARIN} \rightarrow \text{MINIAPP}.$$

The response may include signatures, signed transaction blobs, derived addresses, transaction digests, or structured error codes depending on the invoked function. Once MINIAPP receives the

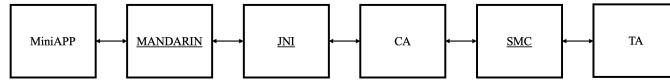


Figure 5: Dataflow

response, it performs the network-facing step—such as broadcasting the signed transaction to an RPC endpoint or propagating it through a peer-to-peer mechanism—while remaining unable to generate signatures independently. This preserves the intended trust division: MINIAPP is permitted to be network-active and user-facing, whereas TA remains the authority for authorization-critical cryptography.

4.4.7 Deterministic error propagation. The symmetry of the forward and reverse paths supports deterministic error handling. Failures occurring at JNI parsing, CA dispatch, secure invocation, or TA execution can be propagated back through the same channel and surfaced in the UI as structured results. Importantly, because module identity is bound to the MINIAPP context, error handling also remains module-scoped: an error from one module cannot be used as a side channel to probe or influence another module’s secure components.

4.5 WalletStore

WalletStore is a vendor-operated backend service that functions as the distribution and authorization hub of the MANDARIN modular wallet ecosystem. While MANDARIN (the system app) provides the on-device runtime and OP-TEE enforces trusted execution for signing, WalletStore operationalizes the module lifecycle at scale by combining (i) a controlled supply chain for module artifacts and (ii) HSM-backed cryptographic provisioning that is bound to device identity.

A central design objective of WalletStore is to ensure that downloadable secure components are delivered in a *device-complete* form. In particular, the TA artifact associated with a module must be returned to the device in a state that is immediately verifiable at load time by the TEE, without requiring any authorization “finalization” step in the REE. This design choice reflects a conservative trust assumption: the REE is treated as a potentially hostile environment for authorization logic, so the server and the secure world together must enforce admission control.

4.5.1 Threat assumptions and trust boundaries. WalletStore is operated by the product vendor and is assumed to be reachable over an authenticated and integrity-protected transport (e.g., HTTPS). However, we explicitly *do not* assume the presence of device attestation, signed nonces, or other hardware-backed request authentication mechanisms in the baseline design. Under this constraint, WalletStore treats the client-provided device identifier primarily as an indexing handle rather than as a cryptographic proof of device identity. Consequently, the system’s core authenticity guarantees are anchored in the TA admission checks performed by OP-TEE rather than in strong client authentication at the server edge.

4.5.2 Core services provided by WalletStore. WalletStore provides three primary functions.

(1) *Module update and download supply chain.* WalletStore hosts and serves versioned module artifacts (MINIAPP, CA, TA) and the associated metadata required for safe installation and update. This includes module identifiers (e.g., UUIDv5), artifact digests, and version compatibility constraints across MINIAPP/CA/TA so that the on-device platform can apply transactional lifecycle operations such as atomic update. From a security standpoint, WalletStore is part of the software supply chain and must therefore be treated as a high-value target; secure update frameworks such as The Update Framework (TUF) provide a useful reference point for designing repository metadata and recovery properties under partial compromise [98, 99]. In addition, supply-chain best-practice frameworks such as SLSA motivate provenance-aware build and release processes to reduce the probability that malicious artifacts are produced and published [100].

(2) *HSM-isolated signing of WalletStore public key under per-device root authority.* WalletStore performs a device-specific delegation step in an isolated HSM environment. Concretely, for each device, WalletStore uses the device’s RootPrivKey to sign WalletStorePubKey, producing a device-scoped WalletStoreSubKey artifact (operationally represented as a .sk value). The RootPrivKey is generated and retained within an HSM during provisioning, while only RootPubKey is embedded per device into the optee_os build. This establishes a delegation link from the device root trust anchor to the WalletStore key domain without exposing root private material outside the HSM boundary. Such an HSM-backed posture aligns with common cryptographic module assurance goals described by standards such as FIPS 140-3 [101].

(3) *Database binding between signed Subkeys and device identifiers (DID).* WalletStore persists the signed WalletStoreSubKey and binds it to a device identifier recorded as a DID. DIDs provide a standardized identifier abstraction that can be resolved into a DID Document containing verification methods and service endpoints [102]. In MANDARIN, the DID is used primarily as a stable database key for retrieving the correct device-specific Subkey during module delivery and update. The choice of DID method and DID-based authentication mechanisms is intentionally left as future work (subsection ??), and we therefore avoid committing to method-specific assumptions in the baseline system description.

4.5.3 Provisioning and delivery workflow. WalletStore’s end-to-end workflow can be described in two phases.

Provisioning phase (pre-deployment)

During provisioning, the vendor’s HSM generates or imports the device’s RootPrivKey and retains it within the HSM boundary. The corresponding RootPubKey is embedded into the device’s optee_os build (per-device embedding). WalletStore then signs WalletStorePubKey under RootPrivKey to create WalletStoreSubKey. The resulting WalletStoreSubKey is stored in the WalletStore database and indexed by the device’s DID. This step is performed prior

to distribution or enrollment so that later module downloads can be satisfied without re-accessing RootPrivKey during routine online operations.

Delivery phase (download/update)

When a device requests a module download or update, WalletStore retrieves the device's WalletStoreSubKey by DID lookup and returns the requested artifacts. Importantly, because the TA must be delivered in a *complete* state, WalletStore composes the final TA package by attaching the appropriate device-specific Subkey material to the TA artifact and completing the required signing steps under WalletStorePrivKey, which is maintained inside a server-side HSM. In the baseline design, TA packaging is sign-only (encryption is not required). This ensures that the device receives a TA artifact that can be admitted by OP-TEE through load-time verification without relying on any authorization step in the untrusted REE.

4.5.4 Operational implications and limitations. The WalletStore design has two notable implications.

First, device-scoped delegation provides strong containment: even if a TA artifact is copied across devices, its authorization chain is anchored in a per-device root domain and is not automatically valid under another device's embedded root. This reduces blast radius under credential compromise and supports per-device governance.

Second, the absence of additional device authentication (e.g., attestation or signed nonces) implies a limitation: a network adversary that can impersonate the device identifier at the application layer may be able to request module artifacts intended for another device. While this does not directly bypass secure-world admission control—the TEE still performs TA verification and enforces the local trust chain—it can increase operational risk (e.g., metadata scraping, abusive download patterns, or unintended exposure of device-targeted artifacts). Strengthening request authentication and DID proof-of-control is therefore a natural extension point and is deferred to future work.

4.5.5 Summary. In summary, WalletStore is a vendor-operated server that provides (i) a controlled module update and download supply chain, (ii) HSM-isolated signing in which each device's RootPrivKey signs WalletStorePubKey to generate a device-specific WalletStoreSubKey, and (iii) persistent storage binding Subkeys to device identifiers (DID) for scalable lookup and lifecycle management. This server-side control plane complements the on-device enforcement plane: WalletStore governs what is distributed, while OP-TEE governs what is admitted for trusted execution.

5 Implementation

We implemented our solution on an STM32MP257F-EV1 evaluation board, featuring a dual-core Arm Cortex-A35 processor. The board runs a custom Android OS built from AOSP with OP-TEE integrated as the secure-world OS (using **ST's OpenSTDroid distribution**). Arm TrustZone allows OP-TEE to run in an isolated Secure World, separated from the normal-world Android system, so that sensitive cryptographic operations execute in a hardware-isolated environment. This architecture mirrors Android's hardware-backed

Keystore approach, where cryptographic keys and operations are handled inside a TEE to limit key exposure.

Within the TEE, we developed a Trusted Application (TA) that holds an ECDSA key pair and provides a signing service. A normal-world client application (CA) communicates with this TA via the GlobalPlatform TEE Client API (libteec), invoking the TA to sign data. To validate the TA's correctness and ensure communication integrity between CA and TA, we implemented a test harness that performs ECDSA signing in both worlds and compares the outputs. In each iteration of this test, the following steps were executed:

- (1) CA generates a random message and key, then computes its ECDSA signature using a standard software library in the normal world.
- (2) CA sends the same message to the TA in the secure world, requesting an ECDSA signature.
- (3) TA signs the message like CA and returns the signature to the CA.
- (4) CA verifies that the TA's signature matches the signature it computed in step 1.

This procedure was repeated for 1000 iterations. All 1000 trials succeeded without any discrepancy: each TA-generated signature exactly matched the CA's result. This outcome confirms that the secure-world signing operates correctly and that the CA-TA data exchange remained intact. Overall, the successful test demonstrates the reliability of our TEE-based implementation for secure key operations.

6 Future Works

HSM-Based Root Key Management Enhancement

The wallet module management architecture proposed in this study establishes a structure that minimizes private-key exposure through user key generation and vendor TA signing mechanisms, while ensuring module integrity by leveraging the OP-TEE environment. However, to further enhance the system's security at a more fundamental level, a more robust approach to managing the root-level signing key used by the wallet store is required. Although this paper employs a software-based method for generating the wallet store's private/public key pair, future research should consider migrating the generation and storage of these keys to a dedicated Hardware Security Module (HSM).

An HSM is a hardware-based cryptographic device certified under FIPS 140-2/3, equipped with a high-quality True Random Number Generator (TRNG) capable of producing keys with high entropy, while preventing key material from being exposed outside the module. In particular, HSMs internally manage root keys and Key Encryption Keys (KEKs), supporting hierarchical key structures that allow upper-level keys to encrypt subordinate keys. This property offers significant advantages in key lifecycle management and key rotation policies. Therefore, future research should focus on generating the wallet store's private key within an HSM and ensuring that the key never leaves the module, thereby further strengthening trust in the current architecture.

Moreover, adopting an HSM enables the following extensions:

- (1) **High-entropy key generation and randomness validation**
TRNG-based key generation achieves higher entropy than

software RNGs and supports statistical randomness validation. This significantly reduces risks associated with brute-force attacks and side-channel analysis targeting the wallet store's root key.

(2) **Administrator-based key ceremony implementation**

Whereas the current system relies on software-based access control for managing the wallet store's private key, an HSM environment enables M-of-N key ceremonies that support distributed administrator control. This minimizes insider threats and eliminates single-point compromise risks.

(3) **Shamir Secret Sharing-based backup and key recovery**

HSMs provide secure mechanisms to distribute and recover encrypted root keys, enabling safe key reissuance and data recovery in case of wallet store decommissioning or failure.

(4) **Integration of secure pipelines between HSM and OP-TEE**

Future work may explore a dual-trust architecture where:
HSM → root key generation
OP-TEE → module verification and execution

Such an approach would extend the trust foundation of TAs executing in the secure world by leveraging HSMs as the root of trust.

(5) **Key rollover and replacement policy design**

Although this study does not detail key rotation intervals, HSMs can automate the deprecation of old keys and activation of new ones during rollover events, enhancing lifecycle security in long-term operation.

(6) **Building cross-vendor trust in multi-vendor environments**

As HSMs support PKI-based authentication structures, the wallet store may function as a central root CA in a multi-vendor TA ecosystem. This allows for unified signing policies and standardized key verification models.

In summary, expanding to an HSM-based key generation and storage system enhances the foundational trustworthiness of the architecture and improves resilience against insider threats, key leakage, and long-term key management issues. Furthermore, integrating HSMs enables the wallet store's keys to evolve beyond simple signing roles and act as a root-of-trust for the entire system. Accordingly, future research should aim to integrate HSM-based key management and establish secure linkage with OP-TEE-based TA verification pipelines, thereby constructing a more complete and secure decentralized modular wallet architecture.

References

- [1] B. Magazine, "Mt. gox files for bankruptcy protection," 2014, accessed: 2025-01-01. [Online]. Available: <https://bitcoinmagazine.com/articles/mt-gox-files-bankruptcy-protection>
- [2] W. S. Journal, "How mt. gox lost nearly half a billion dollars," 2014, accessed: 2025-01-01. [Online]. Available: <https://www.wsj.com/articles/SB10001424052702303801304579410010379087576>
- [3] CipherTrace, "Cryptocurrency exchange security report," 2015, whitepaper.
- [4] J. F. S. Agency, "Amendments to japan's payment services act and financial instruments and exchange act," 2017, government report. [Online]. Available: <https://www.fsa.go.jp/>
- [5] Reuters, "Coincheck loses \$530 million in cryptocurrency hack," 2018, accessed: 2025-01-01. [Online]. Available: <https://www.reuters.com/article/technology/tokyo-based-cryptocurrency-exchange-hacked-losing-530-million-nhk-idUSKBN1FF295/>
- [6] N. Foundation, "Official technical analysis of the coincheck nem hack," 2018, archived NEM Foundation statement on Coincheck incident. [Online]. Available: <https://web.archive.org/web/20180202001024/https://blog.nem.io/statement-on-the-coincheck-hack/>
- [7] N. Y. Times, "How ftx went bankrupt," 2022, accessed: 2025-01-01. [Online]. Available: <https://www.nytimes.com/2022/11/18/business/ftx-collapse.html>
- [8] K. R. Administration, "Ftx trading ltd. bankruptcy filings," 2022, bankruptcy proceedings documents. [Online]. Available: <https://restructuring.ra.rollo.com/FTX>
- [9] C. F. T. Commission, "Complaint against samuel bankman-fried and ftx trading ltd," 2022. [Online]. Available: <https://www.cftc.gov/>
- [10] Bloomberg, "Authorities suspect insider role in ftx asset drain," 2022, news report.
- [11] U. Securities and E. Commission, "U.s. sec testimony on ftx collapse and crypto regulation," 2023. [Online]. Available: <https://www.sec.gov/>
- [12] N. Asia, "Dmm bitcoin hit by massive hack, loses over \$300 million," 2024, accessed: 2025-01-01. [Online]. Available: <https://asia.nikkei.com/Business/Finance/DMM-Bitcoin-hit-by-massive-hack>
- [13] J. F. S. Agency, "Financial services agency (fsa) response to dmm bitcoin incident," 2024, government oversight announcement. [Online]. Available: <https://www.fsa.go.jp/>
- [14] CoinDesk, "Japan's dmm bitcoin suffers major breach, investigation underway," 2024, incident report and analysis. [Online]. Available: <https://www.coindesk.com/>
- [15] JPCERT/CC, "Post-breach forensic analysis of dmm bitcoin wallet infrastructure," 2024, technical assessment.
- [16] "2025 bybit hack," Wikipedia, 2025, accessed: 2025-01-01. [Online]. Available: <https://en.wikipedia.org/wiki/Bybit>
- [17] News.Hada.io, "Bybit hacked: \$1.45b stolen via multisig vulnerability linked to safeWallet," 2025, korean security news portal. [Online]. Available: <https://news.hada.io/topic?id=19375>
- [18] F. B. of Investigation, "Fbi private industry notification: Tradertraitor campaign," 2025, government advisory. [Online]. Available: <https://www.ic3.gov/psa/2025/psa250226>
- [19] Cointelegraph, "Bybit launches bounty program following historic hack," 2025, reporting on incident response. [Online]. Available: <https://cointelegraph.com/>
- [20] Chainalysis, "Crypto crime report 2026," 2026, incident statistics and analysis. [Online]. Available: <https://www.chainalysis.com/>
- [21] Y. News, "Upbit suffers 44.5 billion won loss in hot wallet breach," 2025, accessed: 2025-01-01. [Online]. Available: <https://www.yonhapnews.co.kr/>
- [22] K. Internet and S. Agency, "Kisa launches emergency investigation into upbit security breach," 2025, government agency notice. [Online]. Available: <https://www.kisa.or.kr/>
- [23] F. S. Service, "Fss announces review of exchange incident reporting procedures following upbit hack," 2025, regulatory follow-up. [Online]. Available: <https://www.fss.or.kr/>
- [24] CNN, "North korea-linked lazarus suspected in south korea crypto exchange hack," 2025, international media report. [Online]. Available: <https://edition.cnn.com/>
- [25] Dunamu, "Upbit official response and customer compensation announcement," 2025, corporate disclosure. [Online]. Available: <https://upbit.com/>
- [26] Chainalysis, "Crypto crime report 2024," 2024, accessed: 2025-01-01. [Online]. Available: <https://www.chainalysis.com/>
- [27] L. D. Research, "Hardware wallet vulnerabilities and attacks," 2023, security research findings. [Online]. Available: <https://donjon.ledger.com/>
- [28] B. S. Team, "Hot wallet incidents and centralized custody risks," 2023, security advisory. [Online]. Available: <https://www.binance.com/>
- [29] K. S. Labs, "Supply chain attacks and cold wallet threats," 2022, hardware wallet research. [Online]. Available: <https://blog.kraken.com/>
- [30] H. Kellerman *et al.*, "A comprehensive survey of hardware wallet security," *IEEE Communications Surveys & Tutorials*, 2021.
- [31] M. Sabt *et al.*, "Trusted execution environment: Arm trustzone explained," *ACM Computing Surveys*, 2015.
- [32] V. Costan and S. Devadas, "Intel sgx explained," 2016, iACR report. [Online]. Available: <https://eprint.iacr.org/2016/086.pdf>
- [33] L. Chen *et al.*, "Secure key management for bitcoin using intel sgx," 2019.
- [34] T. Khan *et al.*, "Security challenges in multi-chain wallets," 2022. [Online]. Available: <https://arxiv.org/abs/2203.01111>
- [35] T. of Bits, "Cost and limitations of independent security audits in blockchain systems," 2022, security consulting report. [Online]. Available: <https://www.trailofbits.com/>
- [36] F. Research, "Custody in a multi-chain world: Security and interoperability," 2023, industry whitepaper. [Online]. Available: <https://www.fireblocks.com/>
- [37] Linaro, "Op-tee documentation," 2023, accessed: 2025-01-01. [Online]. Available: <https://optee.readthedocs.io/>
- [38] L. Engineering, "Modular tee design in op-tee," 2022, technical design reference. [Online]. Available: <https://optee.readthedocs.io/architecture/>
- [39] F. Research, "Custody in a multi-chain world: Security and interoperability," 2023. [Online]. Available: <https://www.fireblocks.com/>

- [40] G. Security, "Slsa: Supply-chain levels for software artifacts," 2022. [Online]. Available: <https://slsa.dev/>
- [41] T. of Bits Research, "Software artifact provenance and binary verification," 2022. [Online]. Available: <https://www.trailofbits.com/>
- [42] X. He *et al.*, "Reducing tcb for trusted systems: A survey," *IEEE Security & Privacy*, 2018.
- [43] P. Wuille, "Bitcoin improvement proposal 32: Hd wallets," 2012. [Online]. Available: <https://github.com/bitcoin/bips>
- [44] A. Antonopoulos, "Mnemonic code for generating deterministic keys," 2013. [Online]. Available: <https://github.com/bitcoin/bips>
- [45] A. S. Research, "Isolation guarantees in trusted execution environments," 2020.
- [46] L. Foundation, "Reproducible builds in linux ecosystem," 2021. [Online]. Available: <https://reproducible-builds.org/>
- [47] N. Cybersecurity, "Rollback attacks in secure environments," 2021. [Online]. Available: <https://csrc.nist.gov/>
- [48] B. Mann, "Architecting a more secure world with isolation and virtualization," Arm Community Blog (Architectures and Processors Blog), Aug. 2018, accessed: 2025-12-21. [Online]. Available: <https://developer.arm.com/community/arm-community-blogs/b/architectures-and-processors-blog/posts/architecting-more-secure-world-with-isolation-and-virtualization>
- [49] B. Ngabonziza, D. Martin, A. Bailey, H. Cho, and S. Martin, "Trustzone explained: Architectural features and use cases," In 2016 IEEE 2nd International Conference on Collaboration and Internet Computing (CIC), 2016, accessed: 2025-12-21. [Online]. Available: <https://sefcom.asu.edu/publications/trustzone-explained-cic2016.pdf>
- [50] M. Doran, "Arm trustzone: Secure your embedded systems," DornerWorks Blog, May 2023, accessed: 2025-12-21. [Online]. Available: <https://www.dornerworks.com/blog/arm-trustzone/>
- [51] J. Wiklander, "Subkeys in op-tee," Linaro Blog, Jan. 2024, accessed: 2025-12-21. [Online]. Available: <https://www.linaro.org/blog/subkeys-in-op-tee-part-1/>
- [52] NVIDIA, "Op-tee: Open portable trusted execution environment," Jetson Linux Developer Guide, 2019, accessed: 2025-12-21. [Online]. Available: <https://docs.nvidia.com/jetson/archives/r38.2.1/DeveloperGuide/SD/Security/OpTee.html>
- [53] OP-TEE Project (Linaro), "Subkeys," OP-TEE Documentation, 2023, accessed: 2025-12-21. [Online]. Available: <https://optee.readthedocs.io/en/latest/architecture/subkeys.html>
- [54] S. Susnjara and I. Smalley, "What is blockchain?" IBM Think Blog, 2023, accessed: 2025-12-21. [Online]. Available: <https://www.ibm.com/think/topics/blockchain>
- [55] D. Yaga, P. Mell, N. Roby, and K. Scarfone, "Blockchain technology overview," NIST Interagency/Internal Report (NISTIR) 8202, 2018, accessed: 2025-12-21. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ir/2018/nist.ir.8202.pdf>
- [56] B. Thompson, "Aosp explained: Inside google's open-source os," FindArticles, 2025, accessed: 2025-12-21. [Online]. Available: <https://www.findarticles.com/aosp-explained-inside-google-open-source-os/>
- [57] Android Open Source Project, "Hardware-backed keystore," Official Documentation, 2025, accessed: 2025-12-21. [Online]. Available: <https://source.android.com/docs/security/features/keystore>
- [58] STMicroelectronics, "Stm32mp257f-ev1 evaluation board: Product overview," ST Product Page, 2025, accessed: 2025-12-21. [Online]. Available: <https://www.st.com/en/evaluation-tools/stm32mp257f-ev1.html>
- [59] M. Gentilal, A. Miller, P. Papadimitratos, and A. S. Lagutin, "Trustzone-backed bitcoin wallet," Proceedings of the 2017 ACM Conference on Computer and Communications Security (CCS), 2017, accessed: 2025-12-21. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3031836.3031841>
- [60] H. Dai, S. Han, Z. Chen, and Y. Zhang, "Sblwt: A secure blockchain lightweight wallet based on trustzone," IEEE Access, 2018, accessed: 2025-12-21. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8412192>
- [61] J. Kim, J. Jung, and S. Lee, "Secure user-friendly blockchain modular wallet design using android & op-tee," arXiv preprint, 2025, accessed: 2025-12-21. [Online]. Available: <https://arxiv.org/pdf/2506.17988>
- [62] Arm, "TrustZone for Armv8-A," <https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Learn%20the%20Architecture/TrustZone%20for%20Armv8-A.pdf>, 2020, accessed: 2025-12-21.
- [63] OP-TEE Project, "Trusted Applications (Architecture)," https://optee.readthedocs.io/en/latest/architecture/trusted_applications.html, 2025, accessed: 2025-12-21.
- [64] Android Developers, "WebView - Native bridges | Security," <https://developer.android.com/privacy-and-security/risks/insecure-webview-native-bridges>, 2024, accessed: 2025-12-21.
- [65] GlobalPlatform, "TEE Client API Specification v1.0," https://globalplatform.org/wp-content/uploads/2010/07/TEE_Client_API_Specification-V1.0.pdf, 2010, accessed: 2025-12-21.
- [66] OP-TEE Project, "GlobalPlatform API," https://optee.readthedocs.io/en/latest/architecture/globalplatform_api.html, 2025, accessed: 2025-12-21.
- [67] —, "Subkeys," <https://optee.readthedocs.io/en/latest/architecture/subkeys.html>, 2025, accessed: 2025-12-21.
- [68] IETF, "RFC 4122: A Universally Unique Identifier (UUID) URN Namespace," <https://datatracker.ietf.org/doc/html/rfc4122>, 2005, accessed: 2025-12-21.
- [69] Arm, "TrustZone for Armv8-A," <https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Learn%20the%20Architecture/TrustZone%20for%20Armv8-A.pdf>, 2020, accessed: 2025-12-21.
- [70] OP-TEE Project, "OP-TEE Documentation," <https://optee.readthedocs.io/>, 2025, accessed: 2025-12-21.
- [71] —, "GlobalPlatform API (TEE Client API)," https://optee.readthedocs.io/en/latest/architecture/globalplatform_api.html, 2025, accessed: 2025-12-21.
- [72] GlobalPlatform, "TEE Client API Specification v1.0," https://globalplatform.org/wp-content/uploads/2010/07/TEE_Client_API_Specification-V1.0.pdf, Jul. 2010, accessed: 2025-12-21.
- [73] STMicroelectronics, "Android-based OpenSTDRoid embedded software," https://wiki.st.com/stm32mpu/wiki/Android-based_OpenSTDRoid_embedded_software, 2025, accessed: 2025-12-21.
- [74] —, "OpenSTDRoid distribution," https://wiki.st.com/stm32mpu/wiki/OpenSTDRoid_distribution, 2025, accessed: 2025-12-21.
- [75] —, "STM32 MPU OpenSTDRoid release note - v5.1.0," https://wiki.st.com/stm32mpu/wiki/STM32_MPU_OpenSTDRoid_release_note_-_v5.1.0, 2025, accessed: 2025-12-21.
- [76] —, "How to build TEE for Android," https://wiki.st.com/stm32mpu/wiki/How_to_build_TEE_for_Android, 2025, accessed: 2025-12-21.
- [77] —, "STM32MP25 microprocessor," https://wiki.st.com/stm32mpu/wiki/STM32MP25_microprocessor, 2025, accessed: 2025-12-21.
- [78] OP-TEE Project, "Trusted Applications (Architecture)," https://optee.readthedocs.io/en/latest/architecture/trusted_applications.html, 2025, accessed: 2025-12-21.
- [79] —, "Subkeys," <https://optee.readthedocs.io/en/latest/architecture/subkeys.html>, 2025, accessed: 2025-12-21.
- [80] IETF, "RFC 4122: A Universally Unique Identifier (UUID) URN Namespace," <https://datatracker.ietf.org/doc/html/rfc4122>, 2005, accessed: 2025-12-21.
- [81] OP-TEE Project, "Trusted Applications (Building and Signing)," https://optee.readthedocs.io/en/latest/building/trusted_applications.html, 2025, accessed: 2025-12-21.
- [82] —, "Trusted Applications (v3.19.0) - Building and Signing," https://optee.readthedocs.io/en/3.19.0/building/trusted_applications.html, 2023, accessed: 2025-12-21.
- [83] —, "sign_encrypt.py (OP-TEE optee_os script)," https://github.com/OP-TEE/optee_os/blob/master/scripts/sign_encrypt.py, 2025, accessed: 2025-12-21.
- [84] J. Wiklander, "Subkeys in OP-TEE part I," <https://www.linaro.org/blog/subkeys-in-op-tee-part-1/>, Jan. 2024, accessed: 2025-12-21.
- [85] —, "Subkeys in OP-TEE part 2," <https://www.linaro.org/blog/subkeys-in-op-tee-part-2/>, Apr. 2024, accessed: 2025-12-21.
- [86] Android Open Source Project, "Privileged permission allowlist," <https://source.android.com/docs/core/permissions/perms-allowlist>, 2025, accessed: 2025-12-21.
- [87] —, "Android permissions," <https://source.android.com/docs/core/permissions>, 2025, accessed: 2025-12-21.
- [88] OP-TEE Project, "Trusted Applications (Building and Signing)," https://optee.readthedocs.io/en/latest/building/trusted_applications.html, 2025, accessed: 2025-12-21.
- [89] Android Developers, "JNI tips," <https://developer.android.com/ndk/guides/jni-tips>, 2025, accessed: 2025-12-21.
- [90] OP-TEE Project, "GlobalPlatform API," https://optee.readthedocs.io/en/latest/architecture/globalplatform_api.html, 2025, accessed: 2025-12-21.
- [91] Android Developers, "Network security configuration," <https://developer.android.com/privacy-and-security/security-config>, 2025, accessed: 2025-12-21.
- [92] —, "Security with network protocols," <https://developer.android.com/privacy-and-security/security-ssl>, 2025, accessed: 2025-12-21.
- [93] —, "Load in-app content in a WebView (WebViewAssetLoader)," <https://developer.android.com/develop/ui/views/layout/webapps/load-local-content>, 2025, accessed: 2025-12-21.
- [94] AndroidX, "WebViewAssetLoader source (androidx.webkit)," <https://android.googlesource.com/platform/frameworks/support/+/refs/heads/androidx-main/webkit/webkit/src/main/java/androidx/webkit/WebViewAssetLoader.java>, 2025, accessed: 2025-12-21.
- [95] Android Developers, "WebView - Native bridges | Security," <https://developer.android.com/privacy-and-security/risks/insecure-webview-native-bridges>, 2024, accessed: 2025-12-21.
- [96] OWASP MAS, "MASTG-TEST-0033: Testing for Java Objects Exposed Through WebViews," <https://mas.owasp.org/MASTG-TEST-0033/>, 2025, accessed: 2025-12-21.
- [97] OP-TEE Project, "Trusted Applications (Architecture)," https://optee.readthedocs.io/en/latest/architecture/trusted_applications.html, 2025, accessed: 2025-12-21.
- [98] The Update Framework (TUF), "The Update Framework: Overview," <https://theupdateframework.io/>, 2025, accessed: 2025-12-21.
- [99] —, "The Update Framework Specification," <https://theupdateframework.github.io/specification/latest/>, 2025, accessed: 2025-12-21.

- [100] SLSA, “Supply-chain Levels for Software Artifacts (SLSA),” <https://slsa.dev/>, 2025, accessed: 2025-12-21.
- [101] National Institute of Standards and Technology (NIST), “FIPS 140-3: Security Requirements for Cryptographic Modules,” <https://csrc.nist.gov/pubs/fips/140-3/final>, 2019, accessed: 2025-12-21.
- [102] W3C, “Decentralized Identifiers (DIDs) v1.0: Core architecture, data model, and representations,” <https://www.w3.org/TR/did-core/>, 2022, accessed: 2025-12-21.